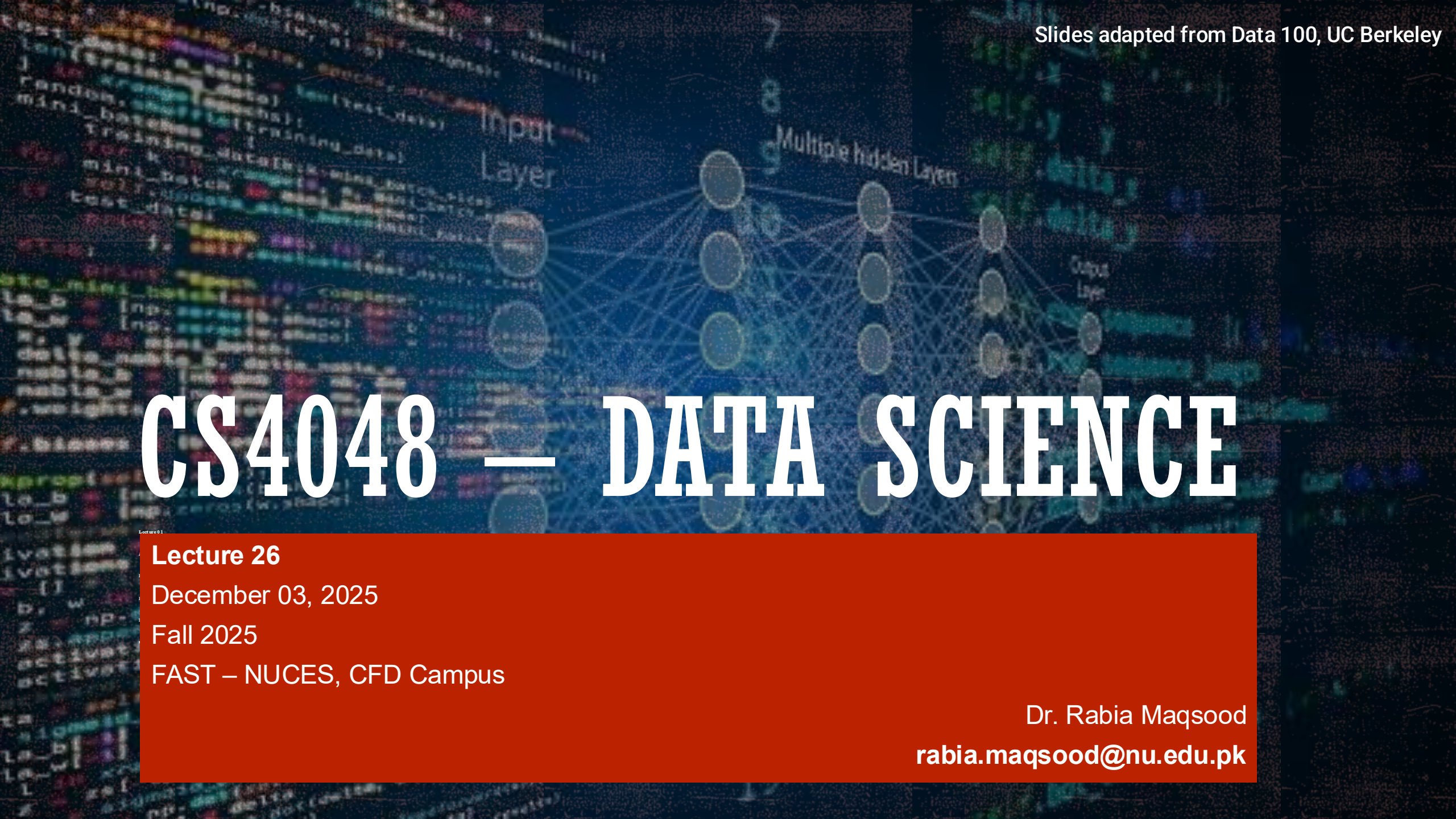


CS4048 — DATA SCIENCE



Lecture 26

December 03, 2025

Fall 2025

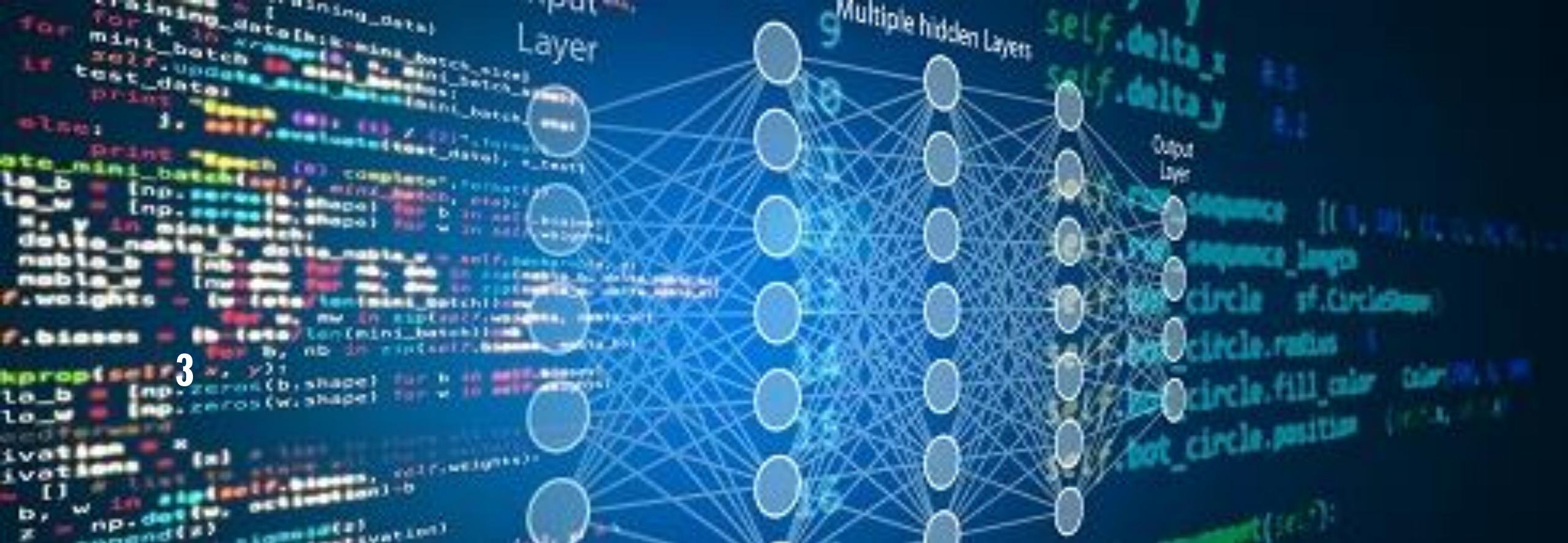
FAST – NUCES, CFD Campus

Dr. Rabia Maqsood

rabia.maqsood@nu.edu.pk

TODAY'S TOPICS

- Logistic Regression
 - Cross entropy (continued)
 - Model fitting
- Evaluation metrics



CROSS-ENTROPY LOSS

THE MODELING PROCESS

Regression ($y \in \mathbb{R}$)

1. Choose a model



Linear Regression

$$\hat{y} = f_{\theta}(x) = x^T \theta$$

2. Choose a loss function

Squared Loss or
Absolute Loss

3. Fit the model

Regularization
Sklearn/Gradient descent

4. Evaluate model performance

R^2 , Residuals, etc.

Classification ($y \in \{0, 1\}$)

Logistic Regression

$$\hat{P}_{\theta}(Y = 1|x) = \sigma(x^T \theta)$$

??

Regularization
Sklearn/Gradient descent

??

THE MODELING PROCESS

Regression ($y \in \mathbb{R}$)



1. Choose a model

Linear Regression

$$\hat{y} = f_{\theta}(x) = x^T \theta$$

2. Choose a loss function

Squared Loss or
Absolute Loss

3. Fit the model

Regularization
Sklearn/Gradient descent

4. Evaluate model
performance

R^2 , Residuals, etc.

Classification ($y \in \{0, 1\}$)

Logistic Regression

$$\hat{P}_{\theta}(Y = 1|x) = \sigma(x^T \theta)$$

Can squared (L2) loss still
work?

Regularization
Sklearn/Gradient descent

??



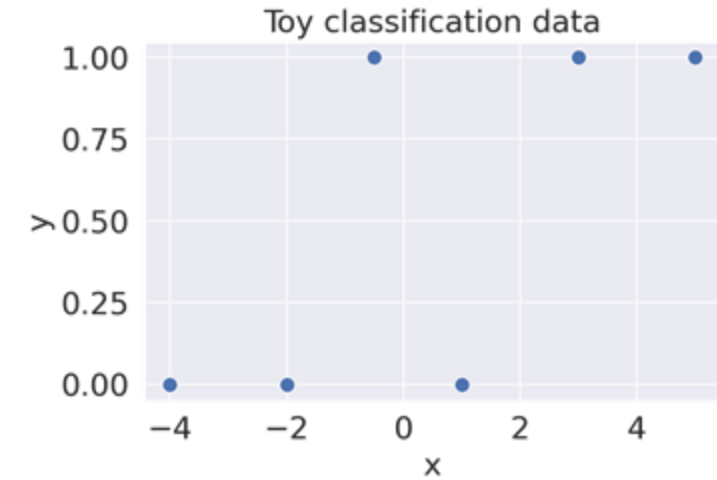
TOY DATASET: L2 LOSS AND COMPUTING OPTIMAL Θ

Logistic Regression model:

$$\hat{P}_{\theta}(Y = 1|x) = \sigma(x^T \theta)$$

Assume no intercept.
So x and θ are scalars.

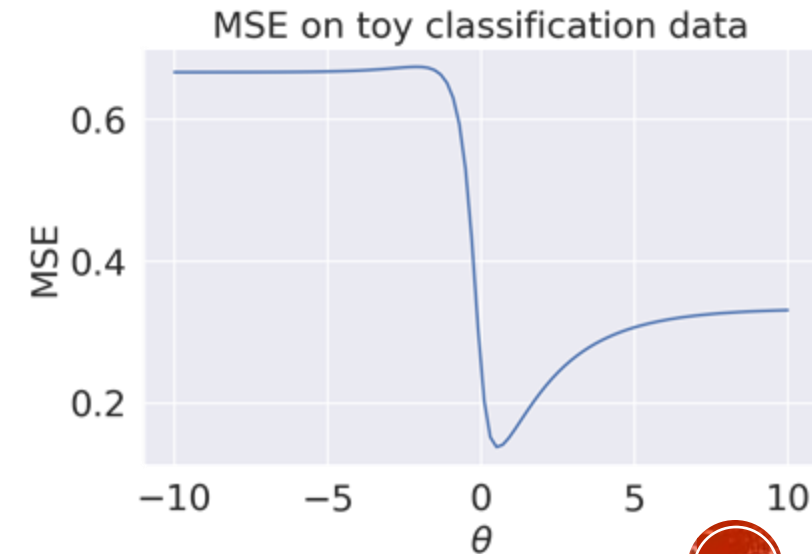
	x	y
0	-4.0	0
1	-2.0	0
2	-0.5	1
3	1.0	0
4	3.0	1
5	5.0	1



Mean Squared Error:

$$R(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - \sigma(x_i^T \theta))^2$$

The MSE loss surface for logistic regression has many issues!



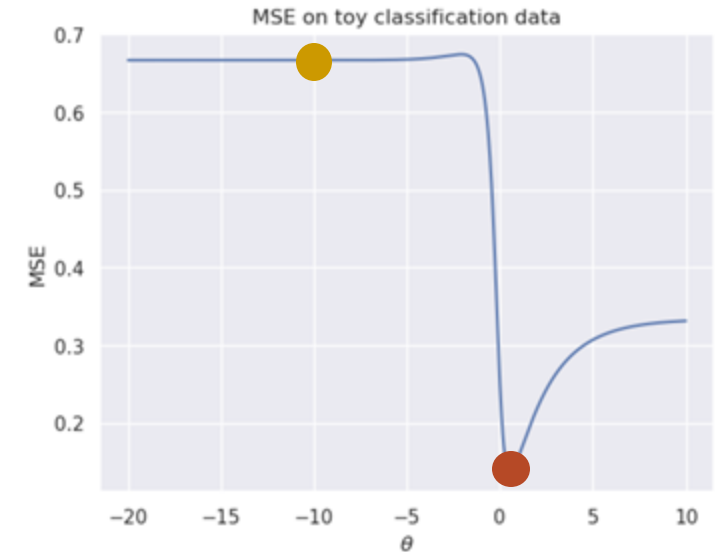
PITFALLS OF SQUARED LOSS

$$R(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - \sigma(x_i^T \theta))^2$$

1. Non-convex. Gets stuck in local minima.

Secant line crosses function, so $R''(\theta)$ is not greater than 0 for all θ .

Gradient Descent: Different initial guesses will yield different optimal estimates.



```
from scipy.optimize import minimize
```

```
minimize(mse_loss_toy_nobias, x0 = 0)["x"][0]
```

0.5446601825581691

```
minimize(mse_loss_toy_nobias, x0 = -5)["x"][0]
```

-10.343653061026611



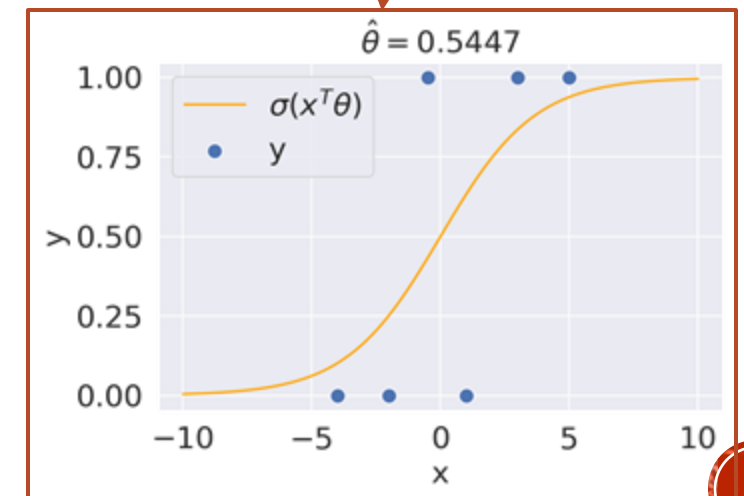
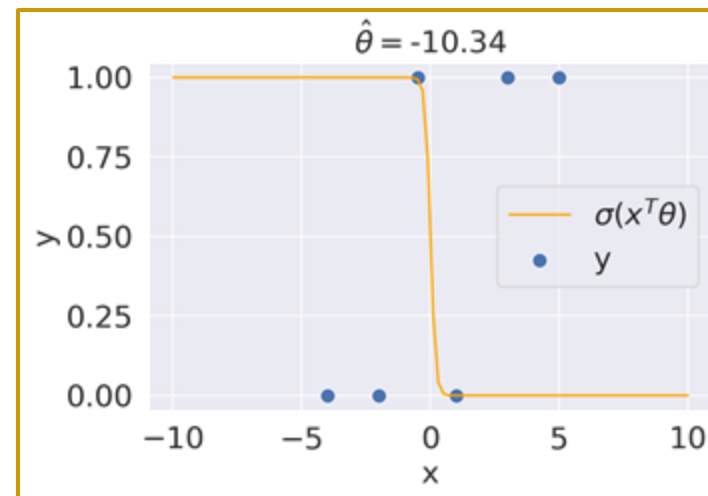
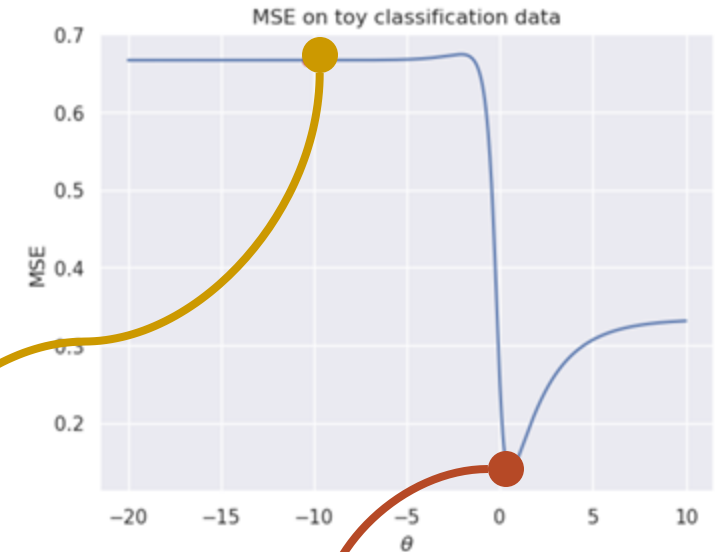
PITFALLS OF SQUARED LOSS

$$R(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - \sigma(x_i^T \theta))^2$$

1. Non-convex. Gets stuck in local minima.

Secant line crosses function, so $R''(\theta)$ is not greater than 0 for all θ .

Gradient Descent: Different initial guesses will yield different optimal estimates.



FOR BINARY CLASSIFICATION, L2 LOSS IS CAPPED AT 1

A logistic regression model outputs the estimated probability of belonging to Class 1.

What is the largest possible L2 loss for a single data point if we use logistic regression?

Solution:

Best case scenario:

- Estimate matches outcome. **L2 loss** = $(0-0)^2 = (1-1)^2 = 0$. Perfect prediction!

Worst case scenarios:

- Estimated probability = 0, Actual outcome = 1. **L2 loss** = $(0-1)^2 = 1$
- Estimated probability = 1, Actual outcome = 0. **L2 loss** = $(1-0)^2 = 1$

So, largest possible L2 loss is **1**. Not a very big penalty for a big error!

PITFALLS OF SQUARED LOSS

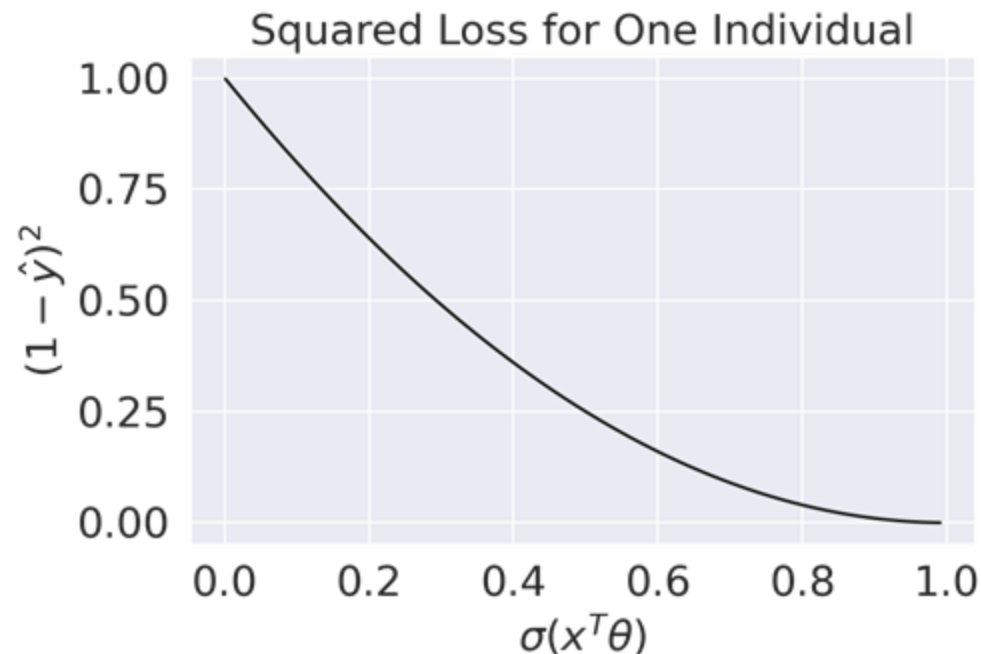
$$R(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - \sigma(x_i^T \theta))^2$$

1. **Non-convex.** Gets stuck in local minima.

2. **Bounded.** Not a good measure of model error.

⌘ We'd like loss functions to penalize “off” predictions.

⌘ MSE never gets very large, because both response and estimated probability are bounded by 1.



If true $y=1$ but
predicted
probability = 0:
 $(y - p)^2 = (1 - 0)^2 = 1$

CHOOSING A DIFFERENT LOSS FUNCTION

Regression ($y \in \mathbb{R}$)

Classification ($y \in \{0, 1\}$)



1. Choose a model

Linear Regression

$$\hat{y} = f_{\theta}(x) = x^T \theta$$

Logistic Regression

$$\hat{P}_{\theta}(Y = 1|x) = \sigma(x^T \theta)$$

2. Choose a loss function

Squared Loss or
Absolute Loss

Cross-Entropy Loss

3. Fit the model

Regularization
Sklearn/Gradient descent

Regularization
Sklearn/Gradient descent

4. Evaluate model
performance

R^2 , Residuals, etc.

??
(next time)

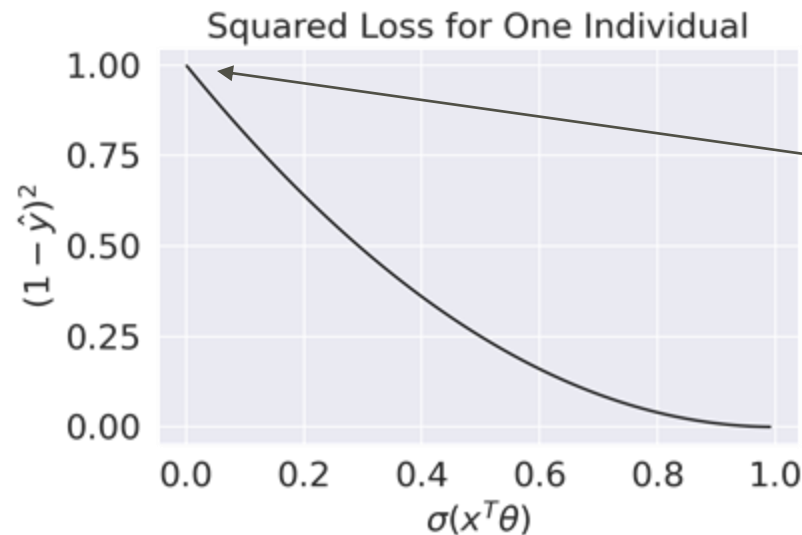
LOSS IN CLASSIFICATION

Let y be a binary label $\{0, 1\}$, and p be the model's estimated probability of Class 1.

In a classification task, how do we want our loss function to behave?

- When the true y is 1, we should incur **high loss** when the model estimates **low p** .
- When the true y is 0, we should incur **high loss** when the model estimates **high p** .

In other words, the behavior of our loss function depends on the value of the true class, y .



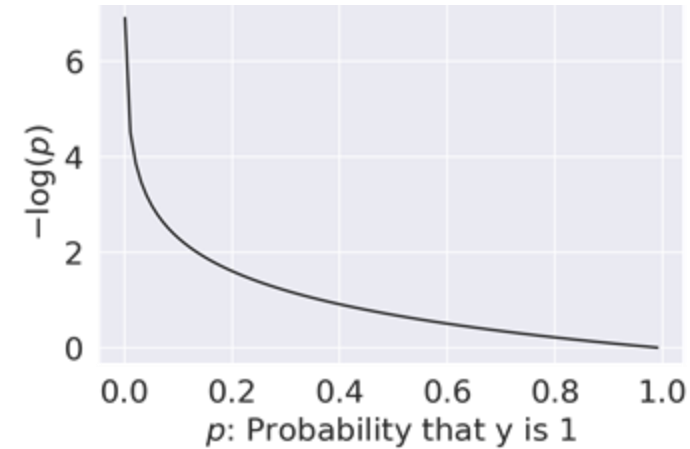
We want a loss function where this is a lot larger than 1!

CROSS-ENTROPY LOSS

Let y be a binary label $\{0, 1\}$, and p be the model's estimated probability of Class 1.

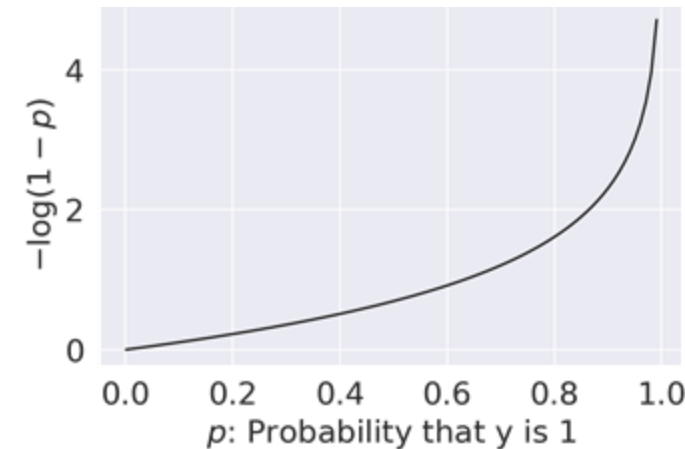
The **cross-entropy (CE) loss** is defined as:

$$\text{CE loss} = \begin{cases} -\log(p), & \text{if } y = 1 \\ -\log(1 - p), & \text{if } y = 0 \end{cases}$$



For $y = 1$,

- $p \rightarrow 0$: ∞ loss
- $p \rightarrow 1$: zero loss



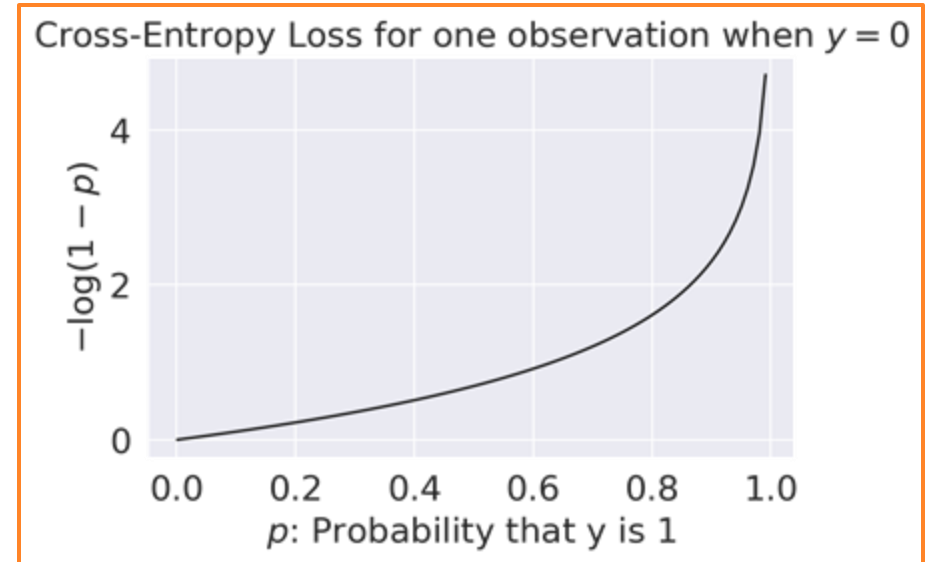
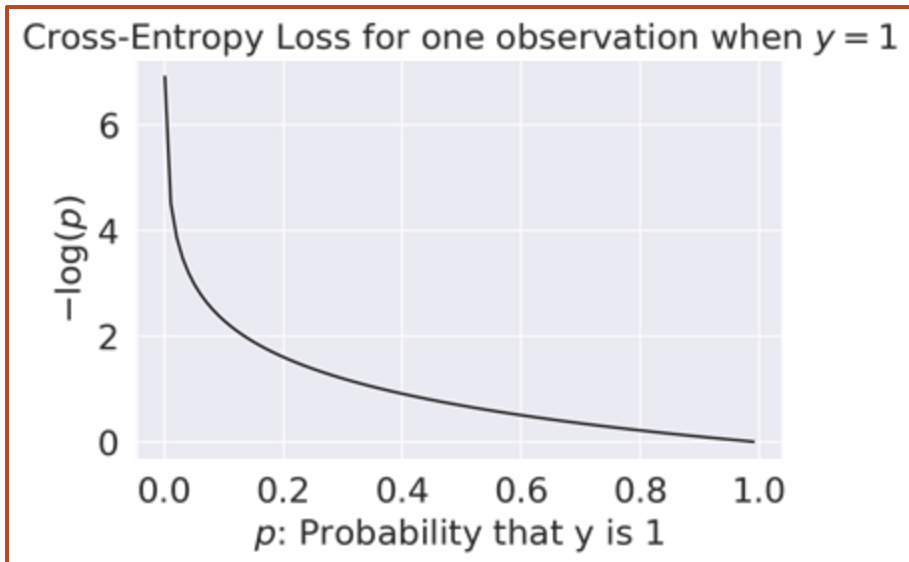
For $y = 0$,

- $p \rightarrow 0$: zero loss
- $p \rightarrow 1$: ∞ loss

CROSS-ENTROPY LOSS: TWO LOSS FUNCTIONS IN ONE!

The Cross-Entropy (CE) loss function is often written more compactly as the **product of the label and the log probability**, summed over both classes.

$$\text{CE loss} = \begin{cases} -\log(p), & \text{if } y = 1 \\ -\log(1 - p), & \text{if } y = 0 \end{cases} \rightarrow - \underbrace{(y \log(p))}_{\text{for } y = \mathbf{1}, \text{ only this term stays}} + \underbrace{(1 - y) \log(1 - p)}_{\text{for } y = \mathbf{0}, \text{ only this term stays}}$$



EMPIRICAL RISK: AVERAGE CROSS-ENTROPY LOSS

For a single datapoint, the cross-entropy (CE) loss is **convex**.

$$\text{CE loss} = - (y \log(p)) + (1 - y) \log(1 - p)$$

What about average CE loss, i.e., empirical risk?

$$p = \hat{P}_\theta(Y = 1|x) = \sigma(x^T \theta)$$

Empirical risk over a sample of size n , for logistic regression:

Estimated $P(Y=1|x)$ from logistic regression

$$\begin{aligned} R(\theta) &= -\frac{1}{n} \sum_{i=1}^n (y_i \log(p_i) + (1 - y_i) \log(1 - p_i)) \\ &= -\frac{1}{n} \sum_{i=1}^n (y_i \log(\sigma(\mathbb{X}_i^T \theta)) + (1 - y_i) \log(1 - \sigma(\mathbb{X}_i^T \theta))) \end{aligned}$$

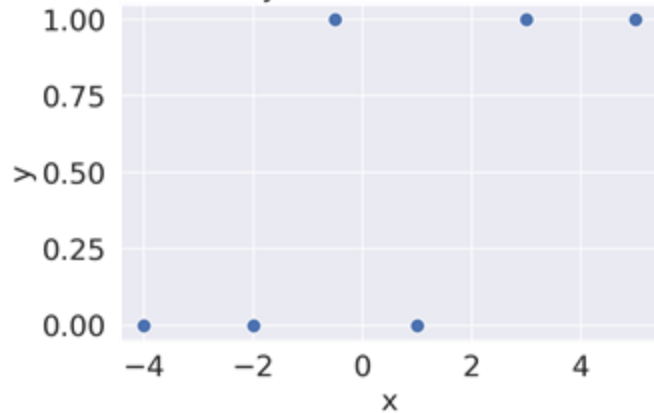
Optimization problem: Find the estimated $\hat{\theta}$ that minimizes $R(\theta)$:

$$\hat{\theta} = \arg \min_{\theta} -\frac{1}{n} \sum_{i=1}^n (y_i \log(\sigma(\mathbb{X}_i^T \theta)) + (1 - y_i) \log(1 - \sigma(\mathbb{X}_i^T \theta)))$$

CONVEXITY PROOF BY PICTURE

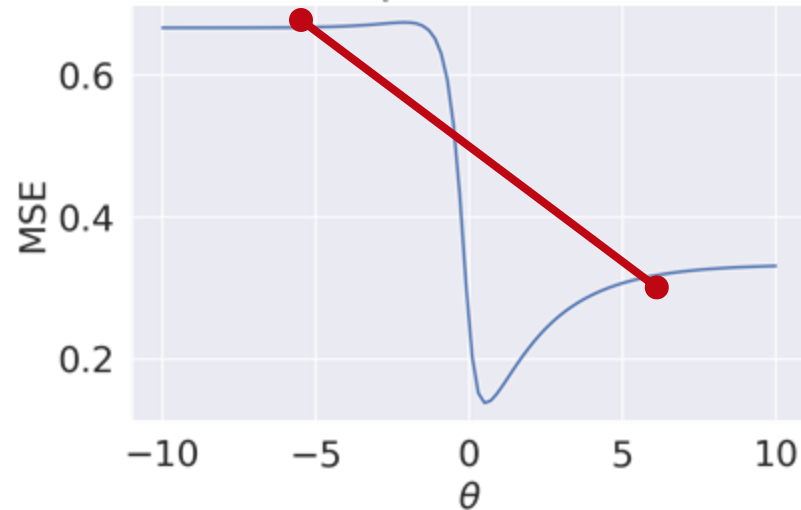
$$\hat{\theta} = \arg \min_{\theta} -\frac{1}{n} \sum_{i=1}^n (y_i \log (\sigma (\mathbb{X}_i^T \theta)) + (1 - y_i) \log (1 - \sigma (\mathbb{X}_i^T \theta)))$$

Toy classification data



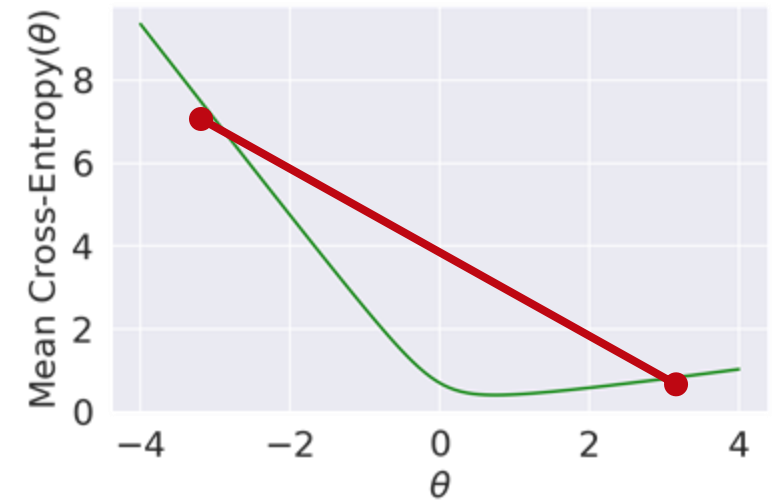
	x	y
0	-4.0	0
1	-2.0	0
2	-0.5	1
3	1.0	0
4	3.0	1
5	5.0	1

Squared Loss Surface



A straight line crosses the curve Non-convex

Cross-Entropy Loss Surface



Convex!

WE DID IT!

1. Choose a model



2. Choose a loss function



3. Fit the model

4. Evaluate model performance

Regression ($y \in \mathbb{R}$)

Linear Regression
 $\hat{y} = f_{\theta}(x) = x^T \theta$

Squared Loss or
Absolute Loss

Regularization
Sklearn/Gradient descent

R^2 , Residuals, etc.

Classification ($y \in \{0, 1\}$)

Logistic Regression
 $\hat{P}_{\theta}(Y = 1|x) = \sigma(x^T \theta)$

Average Cross-Entropy Loss
$$-\frac{1}{n} \sum_{i=1}^n (y_i \log(\sigma(X_i^T \theta)) + (1 - y_i) \log(1 - \sigma(X_i^T \theta)))$$

Regularization
Sklearn/Gradient descent

??



CS4048 - Fall 2025

RECALL: APPLYING A DECISION RULE

Logistic regression estimates the probability that a datapoint belongs to Class 1.

$$\hat{P}_{\theta}(Y = 1|x) = \frac{1}{1 + e^{-x^T \theta}}$$

The model itself does not **classify** (i.e, choose whether a prediction is Class 0 or Class 1).

A **decision rule** tells us how to convert model outputs into classifications. It's our decision.

RECALL: THRESHOLDS

We commonly make decision rules by specifying a **threshold**, T . If the predicted probability is greater than or equal to T , predict Class 1. Otherwise, predict Class 0.

$$p = \hat{P}_\theta(Y = 1|x) = \frac{1}{1 + e^{-x^T \theta}}$$

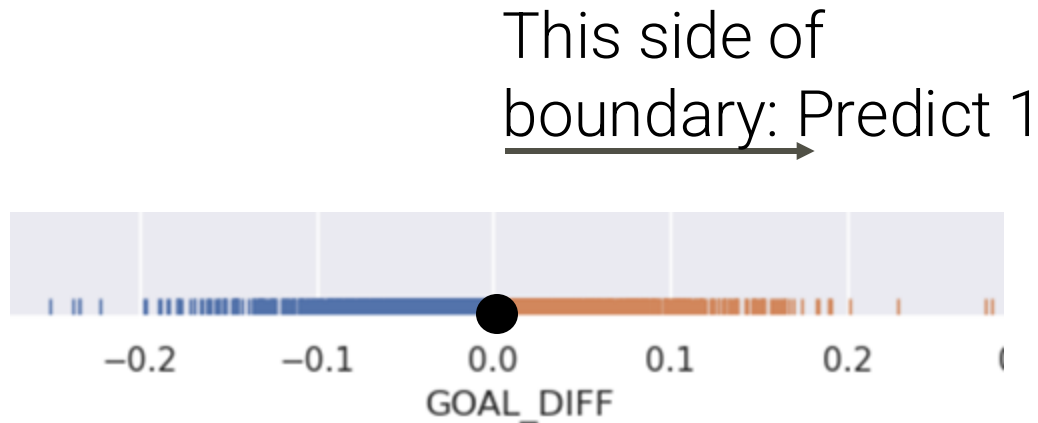
$$\hat{y} = \text{classify}(x) = \begin{cases} \text{Class 1} & p \geq T \\ \text{Class 0} & p < T \end{cases}$$

The threshold is often 0.5, but not always. We'll discuss using $T \neq 0.5$ later today!

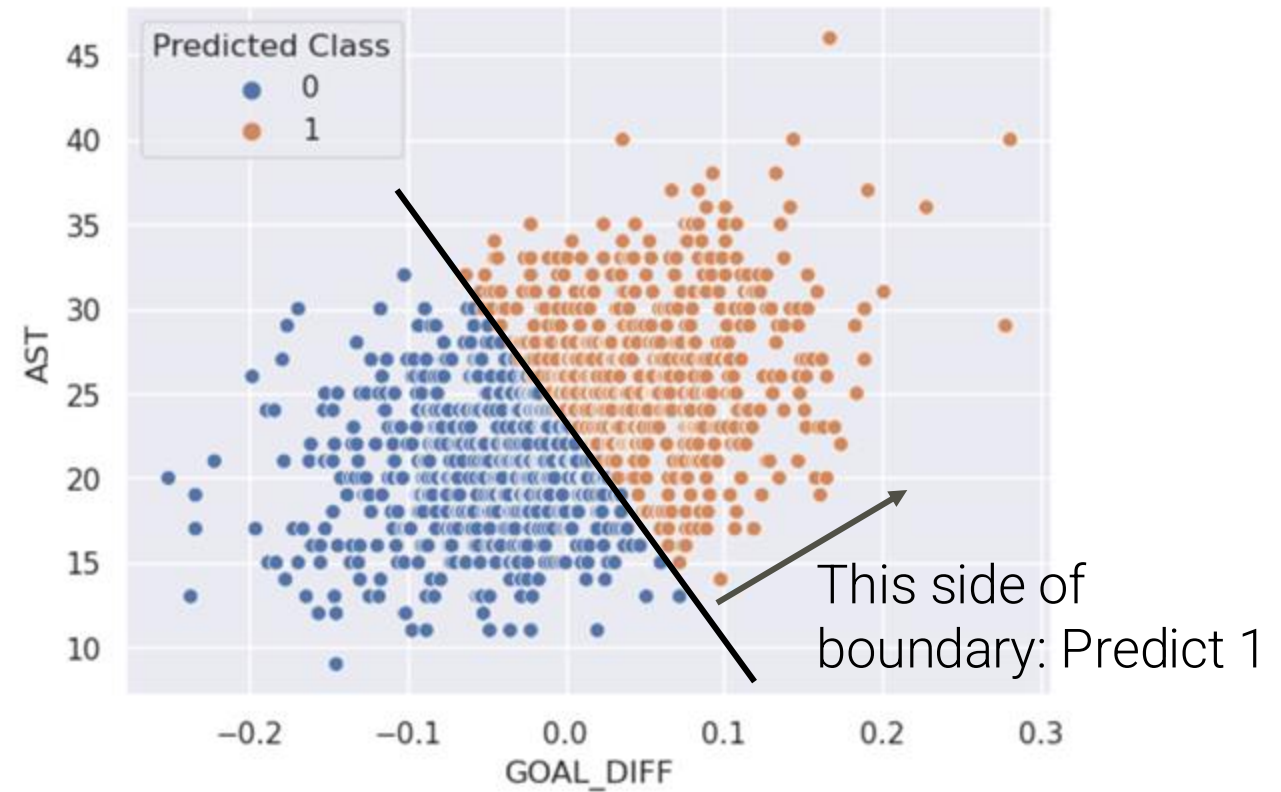
DECISION BOUNDARIES

A **decision boundary** splits the data into predicted classes based on **features**.

- For logistic regression, the decision boundary is a **hyperplane**: a linear combination of the features in p-dimensions.



1 feature: Decision boundary is a 1D point



2 features: Decision boundary is a 2D line

DECISION BOUNDARIES

We can recover the decision boundary from the fitted logistic regression model:

$$\text{Probability threshold} = T = \frac{1}{1 + e^{-(\theta_0 + \theta_1 \times \text{GOAL_DIFF} + \theta_2 \times \text{AST})}}$$

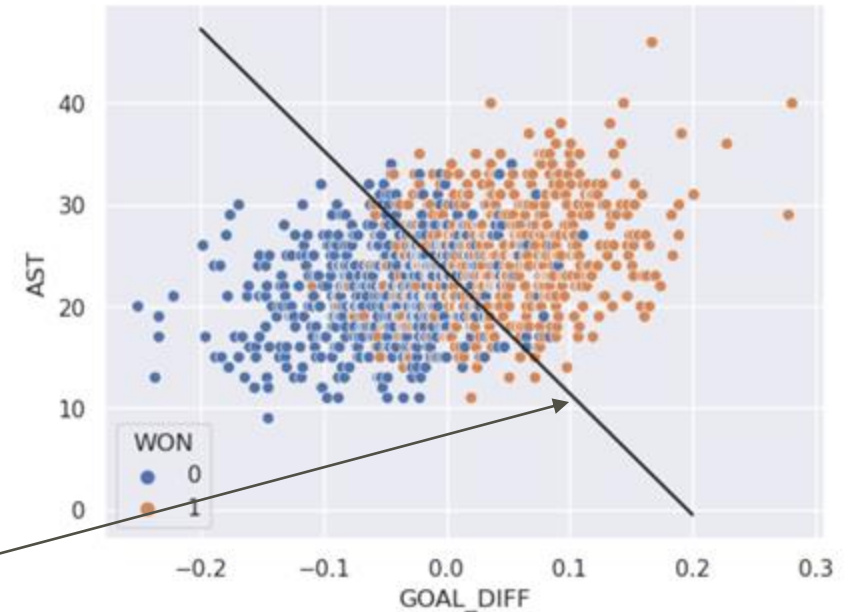
$$\text{Log odds threshold} = \log \frac{T}{1 - T} = \theta_0 + \theta_1 \times \text{GOAL_DIFF} + \theta_2 \times \text{AST}$$

The same equation, just rearranged!

If $T = 0.5$, then the log odds threshold is $\log(0.5/(1-0.5)) = \log(1) = \mathbf{0}$.

$$0 = \theta_0 + \theta_1 \text{GOAL_DIFF} + \theta_2 \text{AST}$$

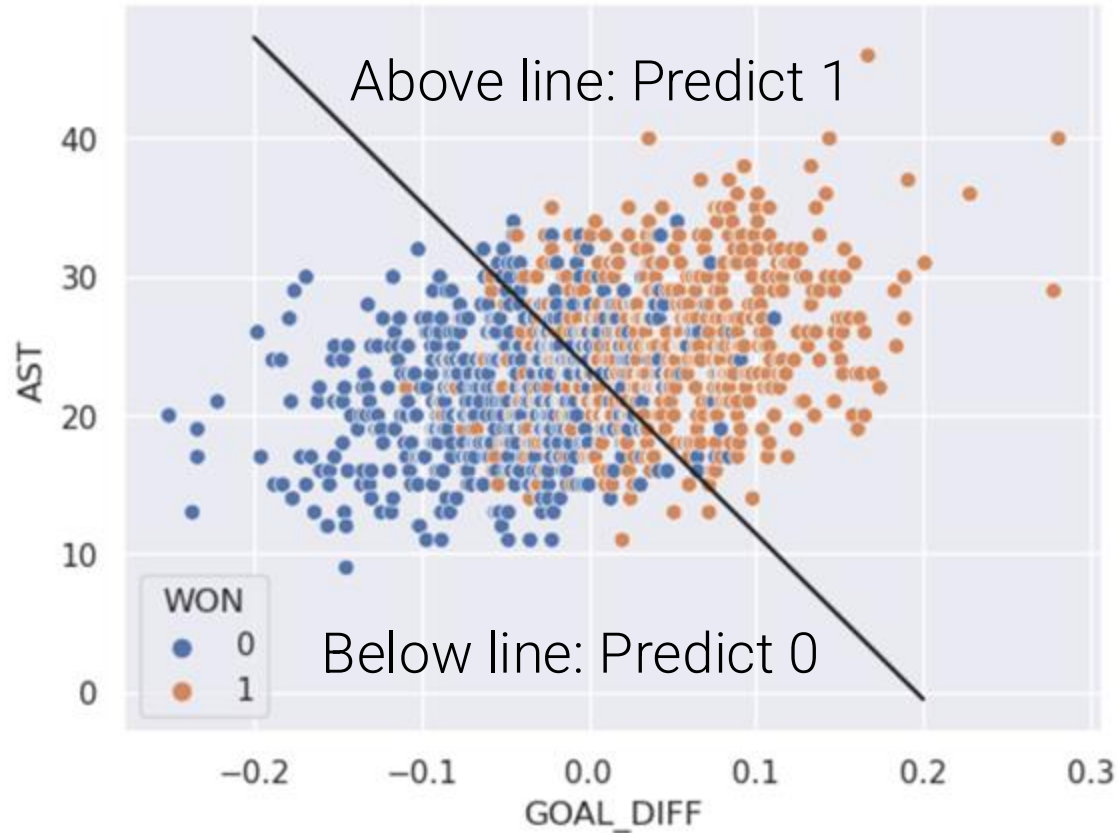
This line is the decision boundary!



The color represents the *true* class, not the model's predictions.

DECISION BOUNDARIES

Our decision boundary in the previous example did not perfectly separate the two classes.



There's a "muddled" region near the decision boundary where our classifier **predicts the wrong class**.

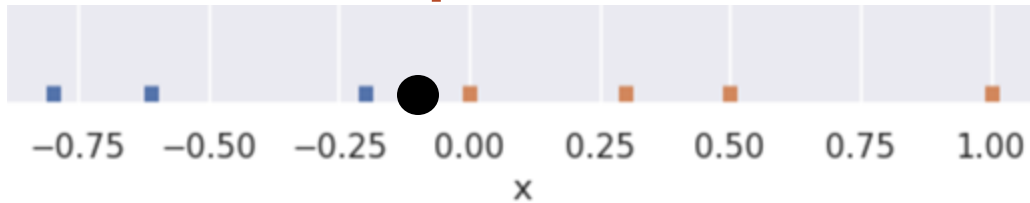
What would the data have to look like for the classifier to make **perfect** predictions?

LINEAR SEPARABILITY

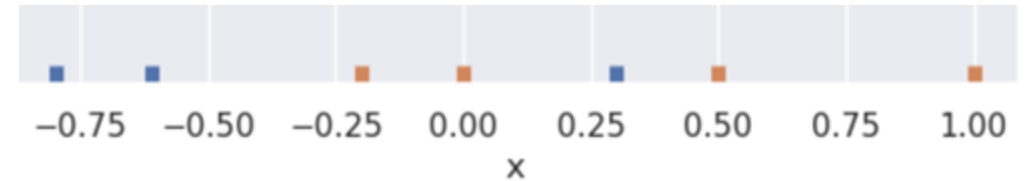
A classification dataset is said to be **linearly separable** if there exists a hyperplane **among input features x** that separates the two classes y .

If there is one feature, look for a point that separates the classes.

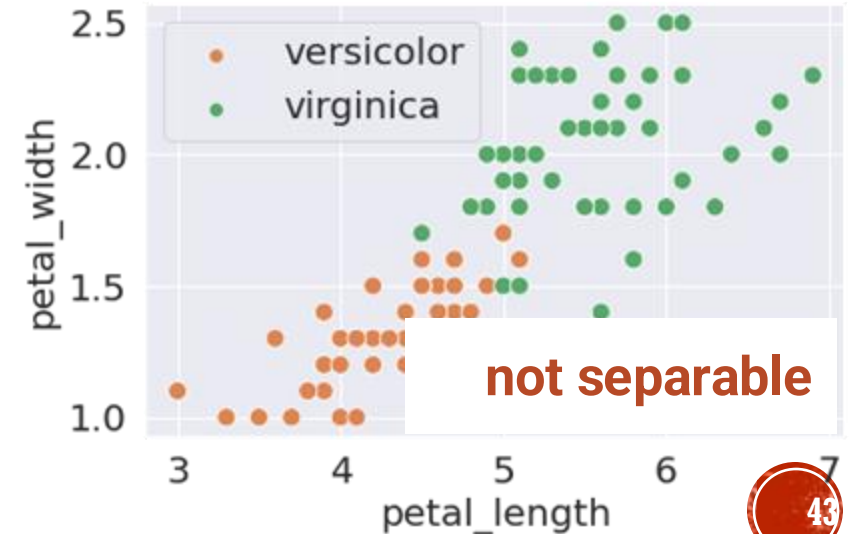
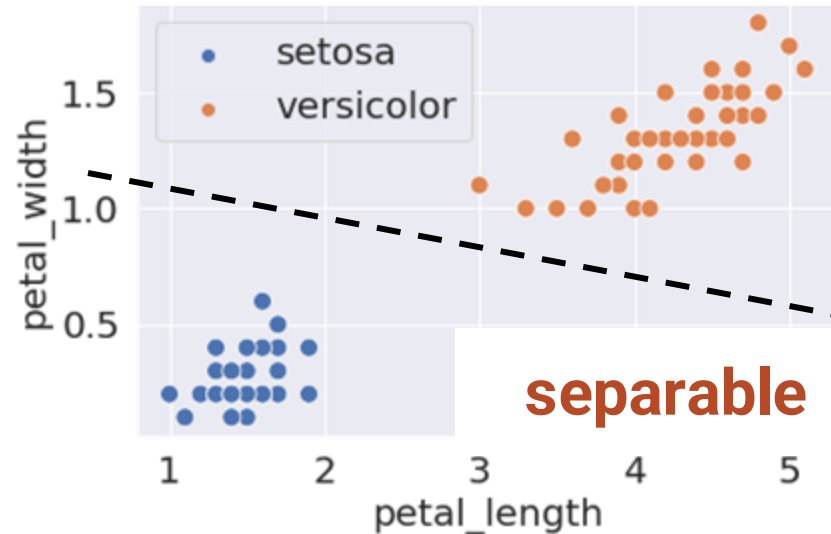
separab



not



If there are two features, look for a line that separates the classes.



LINEAR SEPARABILITY AND LOSS

When the dataset is linearly separable, the logistic regression classifier can **perfectly assign data into classes**.

Can it achieve **0 cross-entropy loss**?

$$-(y \log(p) + (1 - y) \log(1 - p))$$

Loss is 0 if:

- $p = 1$ when $y = 1$
- $p = 0$ when $y = 0$

Consider a simple model with one feature and no intercept.

$$p = \hat{P}_{\theta}(Y = 1|x) = \sigma(\theta x) = \frac{1}{1 + e^{-\theta x}}$$

What θ will achieve 0 loss if we train on the datapoint $x = 1, y = 1$?

LINEAR SEPARABILITY AND LOSS

$$-(y \log(p) + (1 - y) \log(1 - p))$$

Loss is 0 if:

- $p = 1$ when $y = 1$
- $p = 0$ when $y = 0$

Consider a simple model with one feature and no intercept.

$$p = \hat{P}_\theta(Y = 1|x) = \sigma(\theta x) = \frac{1}{1 + e^{-\theta x}}$$

What θ will achieve 0 loss if we train on the datapoint $x = 1, y = 1$?

$$p = 1 = \lim_{\theta \rightarrow \infty} \frac{1}{1 + e^{-\theta * 1}}$$

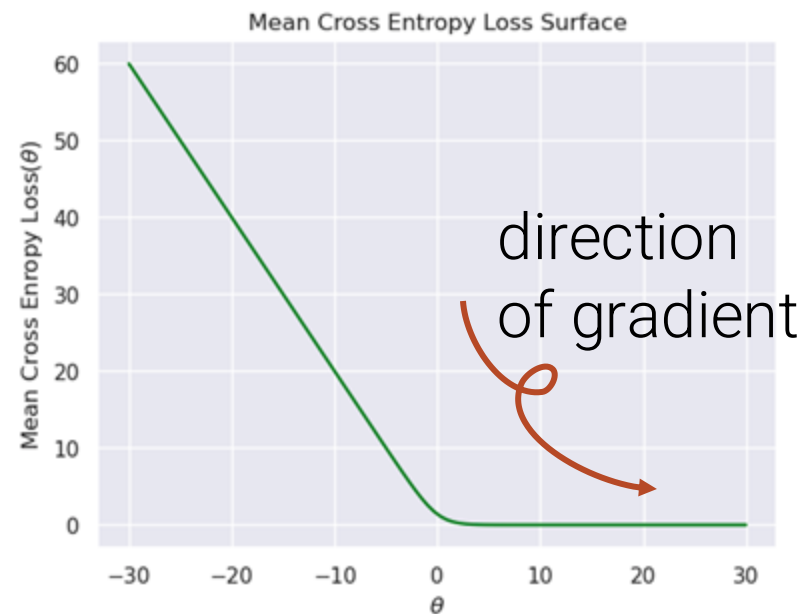
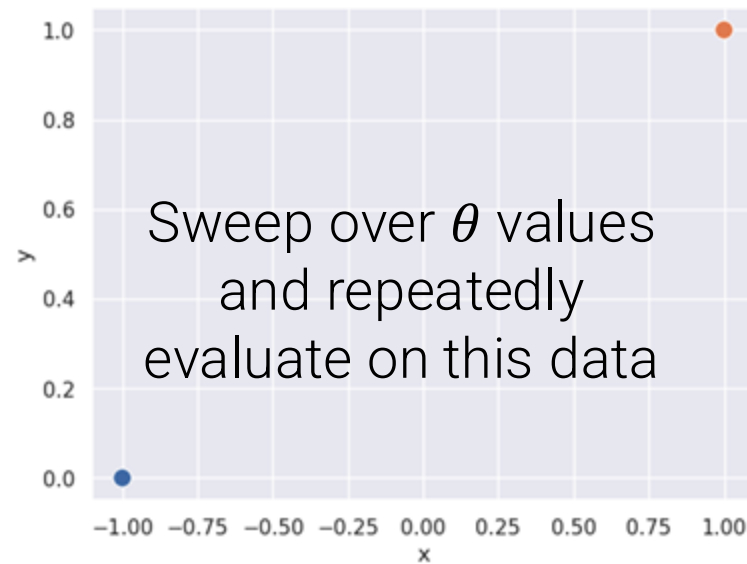
0 loss implies $p = 1$.
Occurs as $\theta \rightarrow \infty$

LINEAR SEPARABILITY CAUSES DIVERGING MODEL PARAMETERS

When data is linearly separable, the **optimal model parameters diverge to $\pm\infty$** .

- The sigmoid can never output exactly 0 or 1, so no finite optimal theta exists.

This can be a problem when using **gradient descent** to fit the model. Consider a simple, linearly separable “toy” dataset with two datapoints:



Impossible to see, but plateau is **slightly tilted downwards**.

Loss approaches 0 as theta increases.

Because gradient descent follows the tilted loss surface downwards, it never converges.

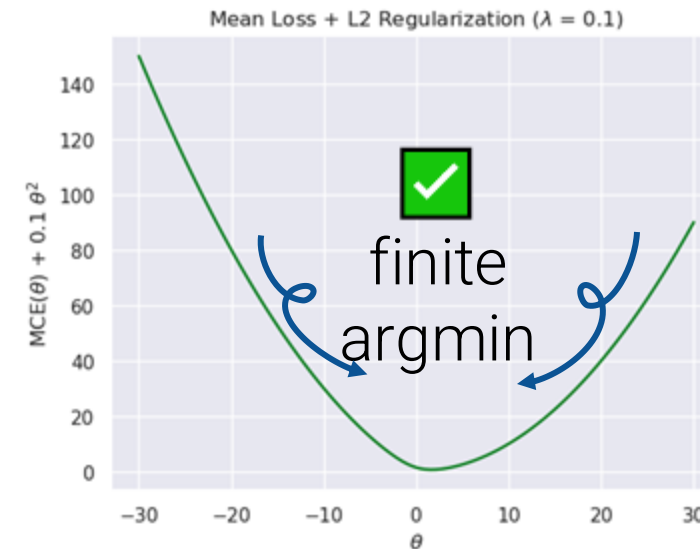
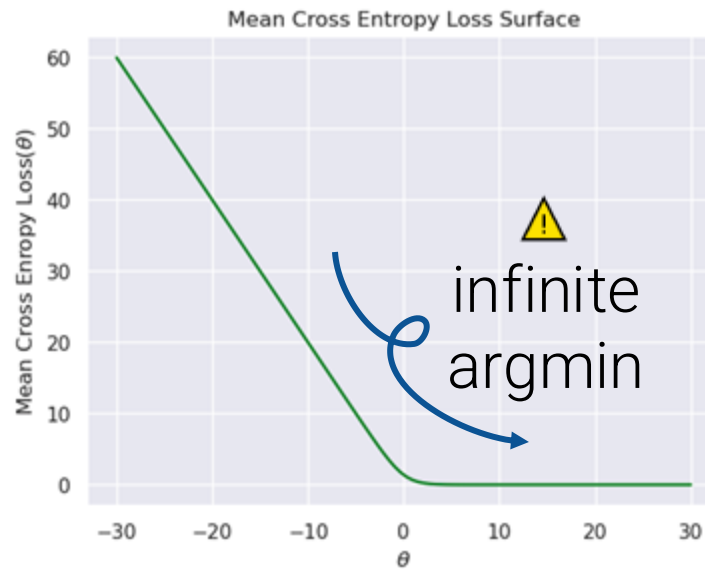
REGULARIZED LOGISTIC REGRESSION

To avoid large weights (particularly on linearly separable data), **use regularization**.

- As with linear regression, standardize features first.

$$\text{Loss} = \text{Logistic Loss} + \lambda \cdot \text{Penalty}$$

$$\operatorname{argmin}_{\theta} - \frac{1}{n} \sum_{i=1}^n (y_i \log(\sigma(X_i^T \theta)) + (1 - y_i) \log(1 - \sigma(X_i^T \theta))) + \lambda \sum_{j=1}^p \theta_j^2$$



```
# sklearn defaults
model = LogisticRegression(
    penalty='l2', C=1.0, ...)
model.fit()
```

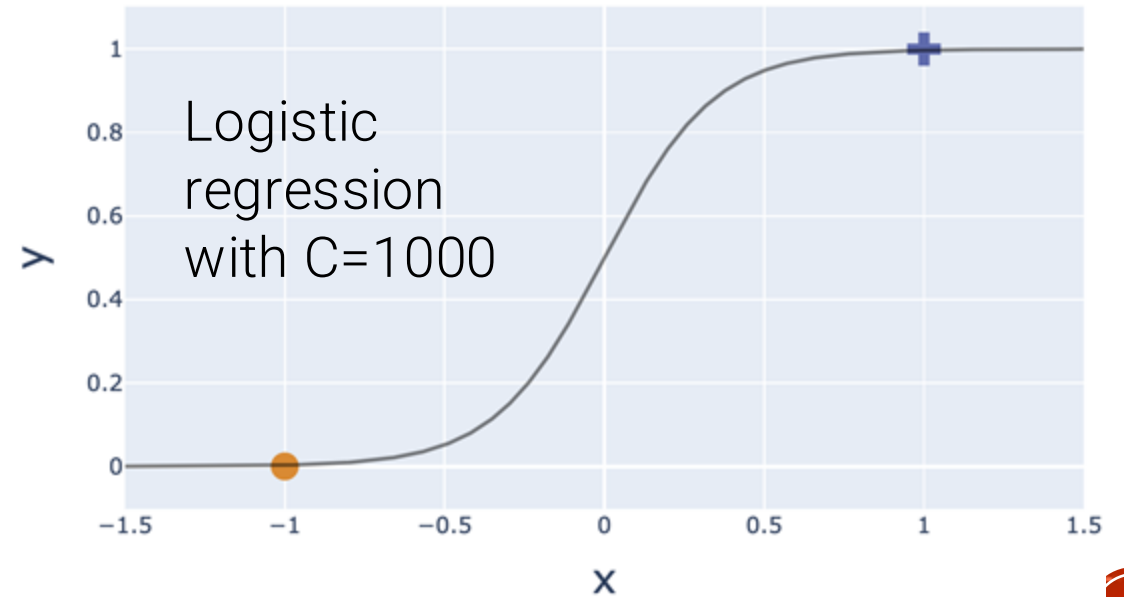
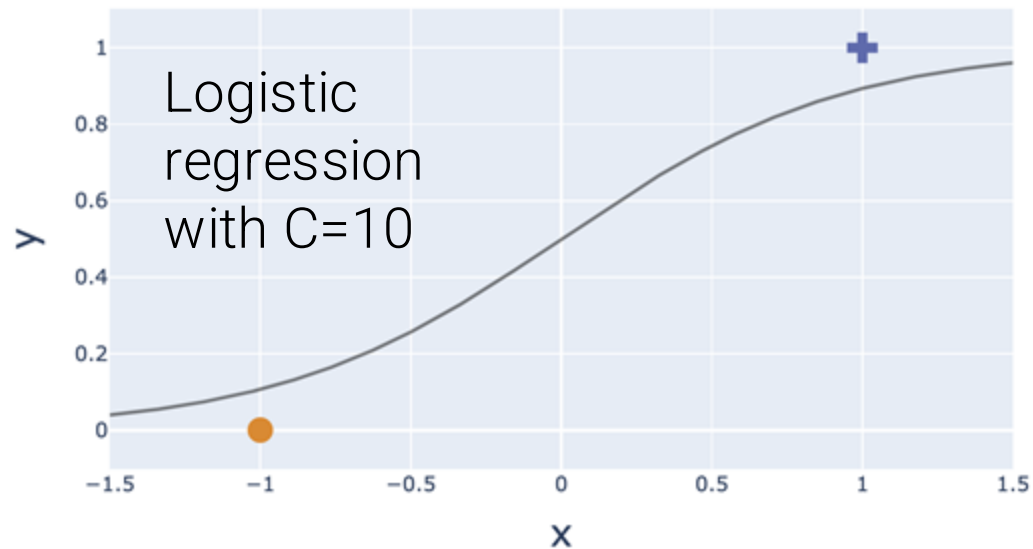
Regularization hyperparameter C is the inverse of λ . **$C = 1 / \lambda$** .

Set C big for minimal regularization, e.g., **$C=300.0$** .

REGULARIZED LOGISTIC REGRESSION

$$\operatorname{argmin}_{\theta} -\frac{1}{n} \sum_{i=1}^n (y_i \log(\sigma(X_i^T \theta)) + (1 - y_i) \log(1 - \sigma(X_i^T \theta))) + \lambda \sum_{j=1}^d \theta_j^2$$

Recall that $\lambda = 1/C$.





PERFORMANCE METRICS

MODELING PROCESS

Regression ($y \in \mathbb{R}$)

1. Choose a model



Linear Regression

$$\hat{y} = f_{\theta}(x) = x^T \theta$$

2. Choose a loss function



Squared Loss or
Absolute Loss

3. Fit the model



Regularization
Sklearn/Gradient descent

**4. Evaluate model
performance**

R^2 , Residuals, etc.

Classification ($y \in \{0, 1\}$)

Logistic Regression

$$\hat{P}_{\theta}(Y = 1|x) = \sigma(x^T \theta)$$

Average Cross-Entropy Loss

$$-\frac{1}{n} \sum_{i=1}^n (y_i \log(\sigma(X_i^T \theta)) + (1 - y_i) \log(1 - \sigma(X_i^T \theta)))$$

sklearn with regularization

Let's do it!

WHY MORE PERFORMANCE METRICS?

Suppose you're an analyst at a real estate startup.

You tell the CEO "Our **linear regression** model is really great! The difference between predicted and actual home prices is typically around \$10,000."

✓ You (effectively) trained the model on **RMSE**, and RMSE is interpretable.

The next day, you tell the CEO "Our **logistic regression** model is really great! When we predict whether a user clicks on a housing ad, the model has an **average cross-entropy loss of 103**."

✗ You trained the model on **CE loss**, but CE loss is not interpretable.

We need different evaluation metrics for interpretable **classification**.

CLASSIFIER ACCURACY

The most basic evaluation metric for a classifier is **accuracy**.

$$\text{accuracy} = \frac{\# \text{ of points classified correctly}}{\# \text{ points total}}$$

If I predict the correct class for 75 out of 100 data points, my accuracy is 75%.

PITFALLS OF ACCURACY: A CASE STUDY

Suppose we're trying to build a classifier to filter spam emails (Project B!).

- Each email is **spam** (1) or **ham** (0).

We have 100 emails: **5** of them are **spam**, and the remaining **95** are real (i.e., **ham**).

Your friend ("Friend 1"):

Classify every email as **ham** (0).

$$\hat{y} = \text{classify}_{\text{friend}}(x) = 0$$

What is the accuracy of your friend's classifier?

$$\text{accuracy} = \frac{\# \text{ of points classified correctly}}{\# \text{ points total}}$$

PITFALLS OF ACCURACY: A CASE STUDY

Suppose we're trying to build a classifier to filter spam emails.

- Each email is **spam** (1) or **ham** (0).

We have 100 emails: **5** of them are **spam**, and the remaining **95** are real (i.e., **ham**).

Your friend ("Friend 1"):

Classify every email as **ham** (0).

$$\text{accuracy}_1 = \frac{95}{100} = 0.95$$

High accuracy...

...but we detected **none** ⚠ of the spam!!!

Your other friend ("Friend 2"):

Classify every email as **spam** (1).

$$\text{accuracy}_2 = \frac{5}{100} = 0.05$$

Low ⚠ accuracy...

...but we detected **all** of the spam!!!

While widely used, the accuracy metric is **is problematic** if there is **class imbalance**.

TYPES OF CLASSIFICATIONS

“**positive**” means a **prediction** of **1**.
“**negative**” means a **prediction** of **0**.

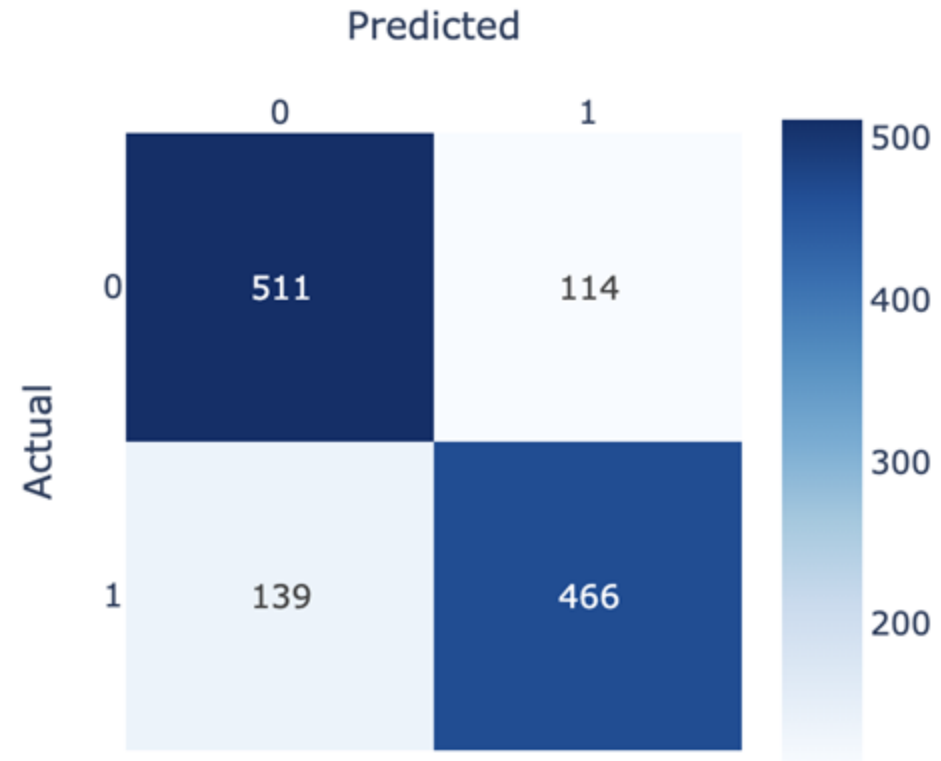
“**True**” means **correct** prediction.
“**False**” means **incorrect** prediction.

		Prediction $\hat{y}$	
		0	1
Actual y	0	True negative (TN)	False positive (FP)
	1	False negative (FN)	True positive (TP)

TYPES OF CLASSIFICATIONS

A **confusion matrix** plots TP, TN, FP, FN for a particular classifier, dataset, and **threshold (!)**.

Our **0/1 predictions** depend on our choice of probability threshold, so the confusion matrix can also **change with a new threshold!**



EFFECT OF CHANGING THE PROBABILITY THRESHOLD ON TP

I **increase** the probability threshold. What happens to the count of **true positives (TP)**?

	Threshold=0.6		Threshold=0.8	
	Threshold=0.3			
Estimated probabilities	0.21	0.63	0.85	
Actual class	0	1	1	
Prediction (T=0.3)	0 (TN)	1 (TP)	1 (TP)	TP = 2
Prediction (T=0.6)	0 (TN)	1 (TP)	1 (TP)	No change!
Prediction (T=0.8)	0 (TN)	0 (FN)	1 (TP)	TP = 1

If we **increase** the threshold, it becomes **harder** to make a positive prediction.

The number of **positives (P)** will **stay the same or decrease**.

So, TP and FP will either go down or stay the same, too.

EFFECT OF CHANGING THE PROBABILITY THRESHOLD ON

FN Reduce the probability threshold. What happens to the count of **false negatives (FN)**?

Threshold=0.3

Threshold=0.7

Threshold=0.8

Estimated probabilities

0.21

0.63

0.85

Actual class

0

1

1

Prediction (T=0.8)

0 (TN)

0 (FN)

1 (TP)

FN = 1

Prediction (T=0.7)

0 (TN)

0 (FN)

1 (TP)

No change!

Prediction (T=0.3)

0 (TN)

1 (TP)

1 (TP)

FN = 0

If we **reduce** the threshold, it becomes **harder** to make a negative prediction.

The number of **negatives (N)** will **stay the same or decrease**.

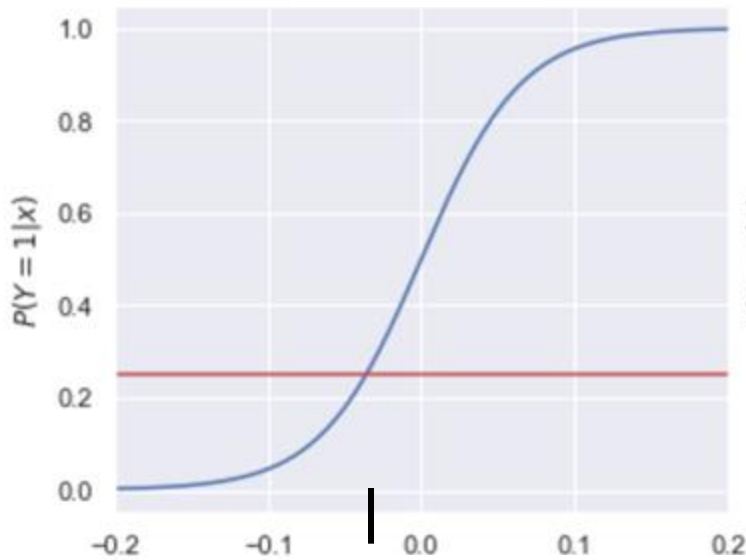
So, TN and FN will either go down or stay the same, too.

CHANGING THE THRESHOLD

$$\hat{y} = \text{classify}(x) = \begin{cases} \text{Class 1} & p \geq T \\ \text{Class 0} & p < T \end{cases}$$

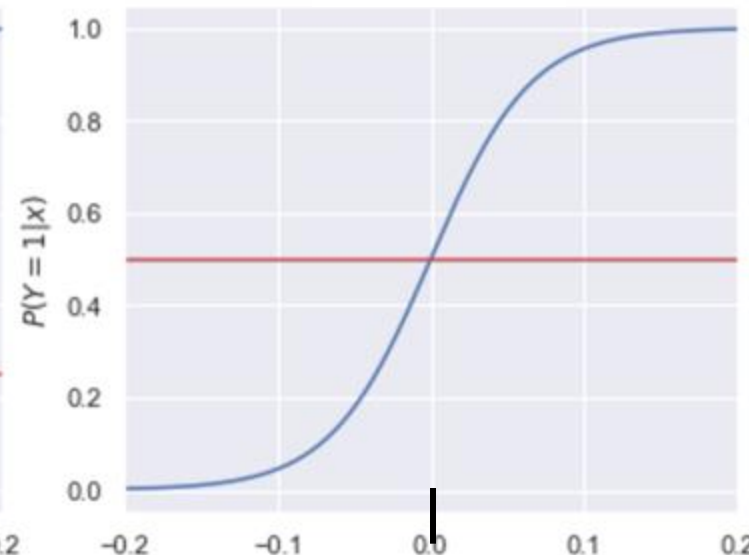
As we increase the threshold T , we “raise the standard” of how confident our classifier needs to be to predict 1 (i.e., “positive”).

$T = 0.25$

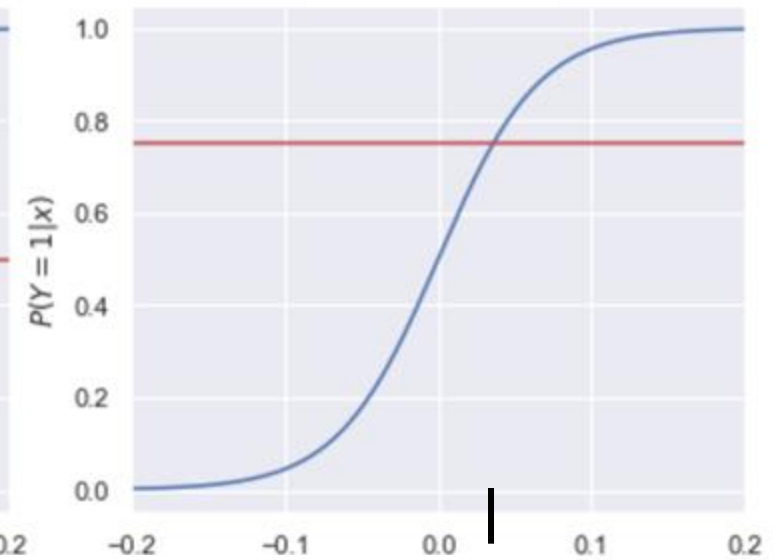


These x will all predict 1

$T = 0.50$



$T = 0.75$



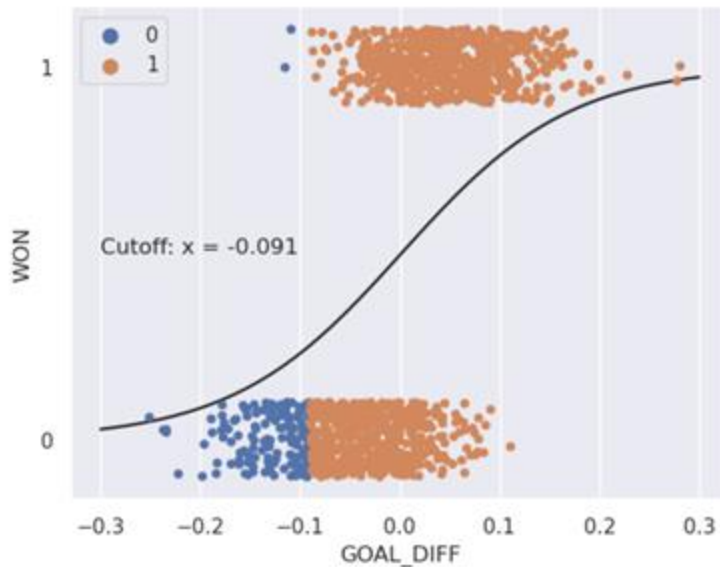
Fewer positives

CHANGING THE THRESHOLD

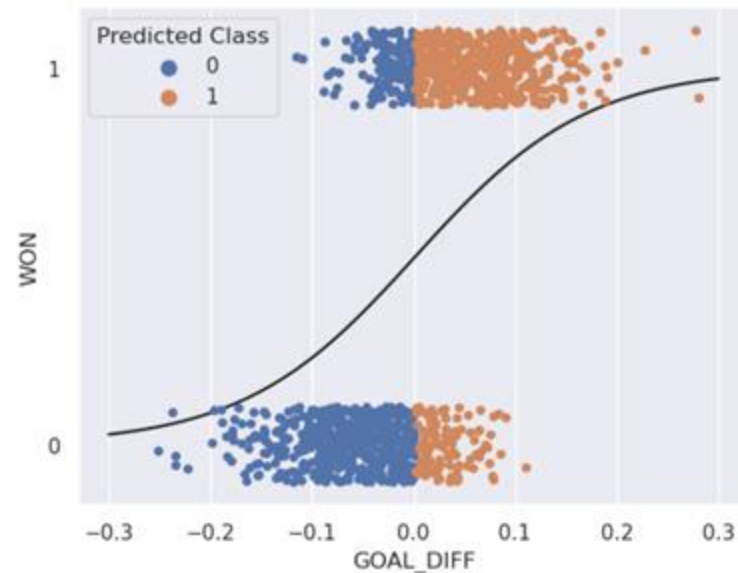
$$\hat{y} = \text{classify}(x) = \begin{cases} \text{Class 1} & p \geq T \\ \text{Class 0} & p < T \end{cases}$$

As we increase the threshold T , we “raise the standard” of how confident our classifier needs to be to predict 1 (i.e., “positive”).

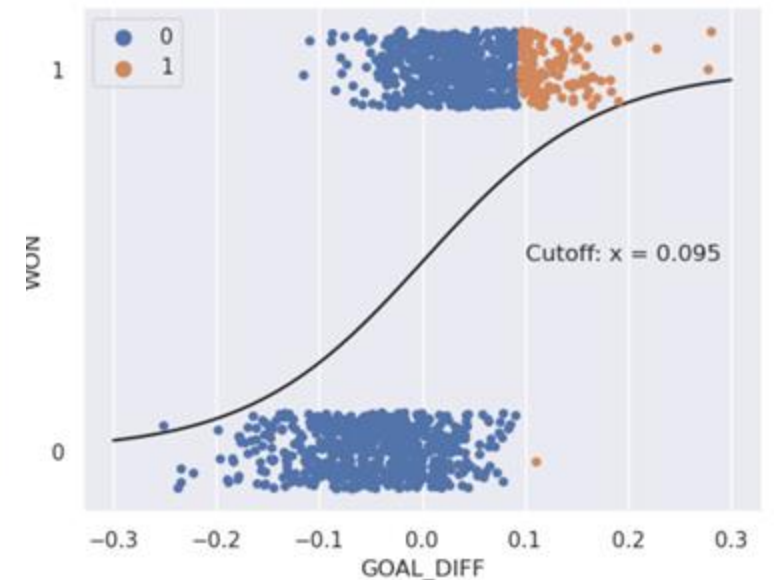
$T = 0.25$



$T = 0.50$

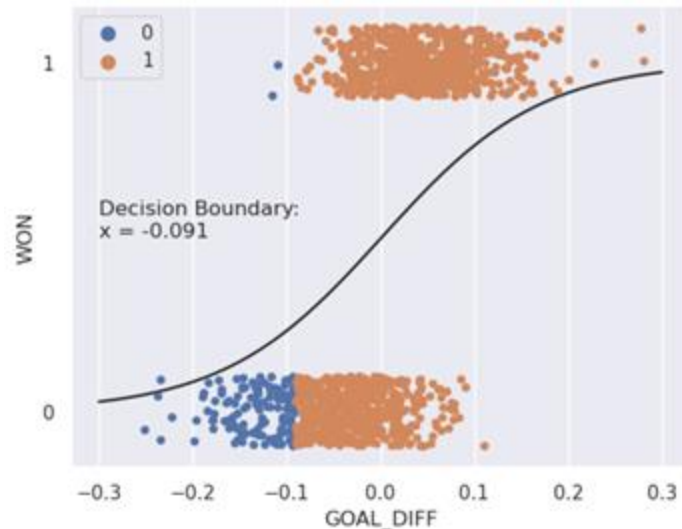


$T = 0.75$

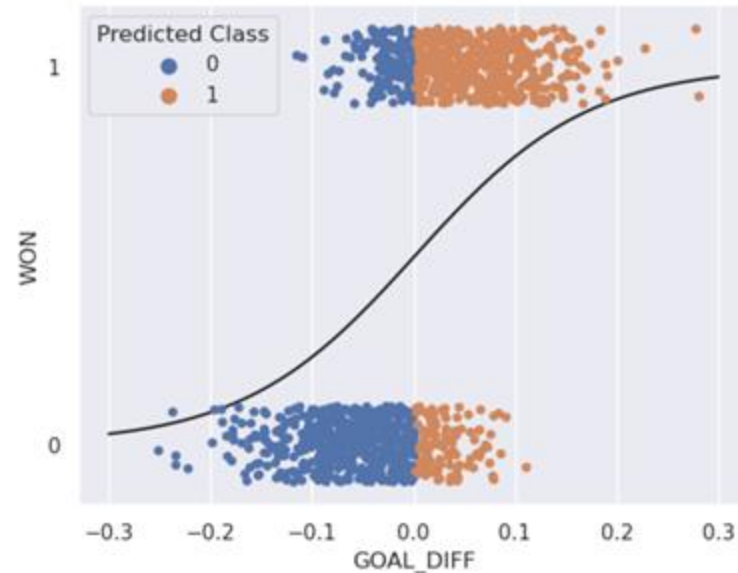


CHANGING THE THRESHOLD

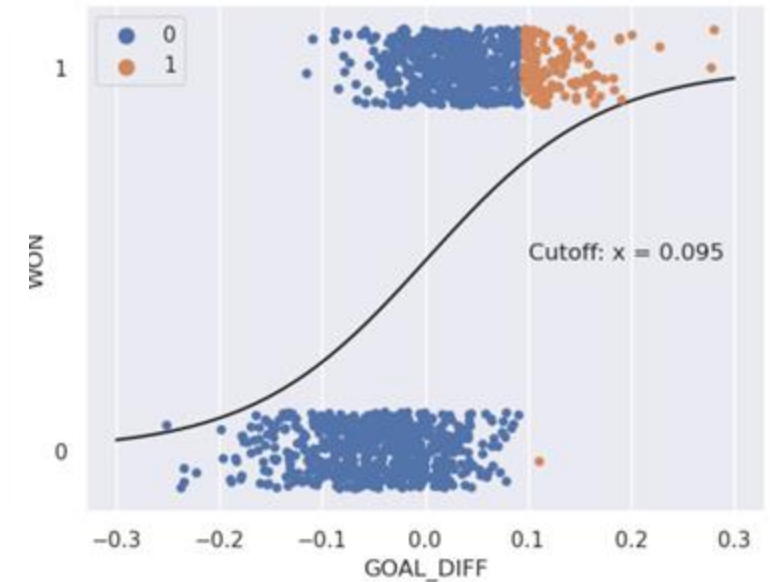
$T = 0.25$



$T = 0.50$



$T = 0.75$



How to interpret a higher classification threshold: the model needs to predict a higher probability of a point belonging to Class 1 before it can confidently classify it as Class 1

- As T increases, we predict fewer positives!

Changing the threshold allows us to fine tune how “confident” we want our model to be before making a positive prediction

ACCURACY, PRECISION, AND RECALL

		Prediction	
		0	1
Actual	0	TN	FP
	1	FN	TP

$$\text{accuracy} = \frac{TP + TN}{n}$$

What proportion of **all points** were correctly classified?

"What % of emails did I classify correctly?"

		Prediction	
		0	1
Actual	0	TN	FP
	1	FN	TP

$$\text{precision} = \frac{TP}{P} = \frac{TP}{TP + FP}$$

Of all **positives (P)**, what proportion were correct (**TP**)?

"What % of spam predictions were correct?"

		Prediction	
		0	1
Actual	0	TN	FP
	1	FN	TP

$$\text{recall} = \frac{TP}{\text{Actual 1s}} = \frac{TP}{TP + FN}$$

Of all **actual 1s**, what proportion did our model **detect**?

"What % of spam emails were detected?"

TP, TN, FP, FN can change with a new threshold, so all of these metrics can change too! These are **threshold-dependent** metrics.

Recall is also known as sensitivity

BACK TO THE SPAM

Suppose we're trying to build a classifier to filter spam emails.

- Each email is **spam** (1) or **ham** (0).

Let's say we have 100 emails, of which only **5** are truly **spam**, and the remaining **95** are **ham**.

Your friend:

Classify every email as **ham** (0).

$$\text{accuracy}_1 = \frac{95}{100} = 0.95$$

$$\text{precision}_1 = \frac{0}{0 + 0} = \text{undefined}$$

$$\text{recall}_1 = \frac{0}{0 + 5} = 0$$

$$\begin{aligned}\text{accuracy} &= \frac{TP + TN}{n} \\ \text{precision} &= \frac{TP}{TP + FP} \\ \text{recall} &= \frac{TP}{TP + FN}\end{aligned}$$

	0	1
0	TN: 95	FP: 0
1	FN: 5	TP: 0

BACK TO THE SPAM

Suppose we're trying to build a classifier to filter spam emails.

- Each email is **spam** (1) or **ham** (0).

Let's say we have 100 emails, of which only **5** are truly **spam**, and the remaining **95** are **ham**.

$$\begin{aligned}\text{accuracy} &= \frac{TP + TN}{n} \\ \text{precision} &= \frac{TP}{TP + FP} \\ \text{recall} &= \frac{TP}{TP + FN}\end{aligned}$$

Your friend:

Classify every email as **ham** (0).

$$\text{accuracy}_1 = \frac{95}{100} = 0.95$$

$$\text{precision}_1 = \frac{0}{0 + 0} = \text{undefined}$$

$$\text{recall}_1 = \frac{0}{0 + 5} = 0$$

Your other friend ("Friend 2"):

Classify every email as **spam** (1).

$$\text{accuracy}_2 = \frac{5}{100} = 0.05$$

$$\text{precision}_2 = \frac{5}{5 + 95} = 0.05$$

$$\text{recall}_2 = \frac{5}{5 + 0} = 1.0$$

	0	1
0	TN: 0	FP: 95
1	FN: 0	TP: 5

} Many false positives!

} No false negatives!

PRECISION VS. RECALL

$$\text{precision} = \frac{TP}{TP + FP}$$

Precision penalizes FPs.

$$\text{recall} = \frac{TP}{TP + FN}$$

Recall penalizes FNs.

There is a **tradeoff** between precision and recall; they are often **inversely related**.

In many settings, FPs and FNs have **different costs**.

- You should **adjust** your threshold to optimize for **overall** costs of FPs and FNs.

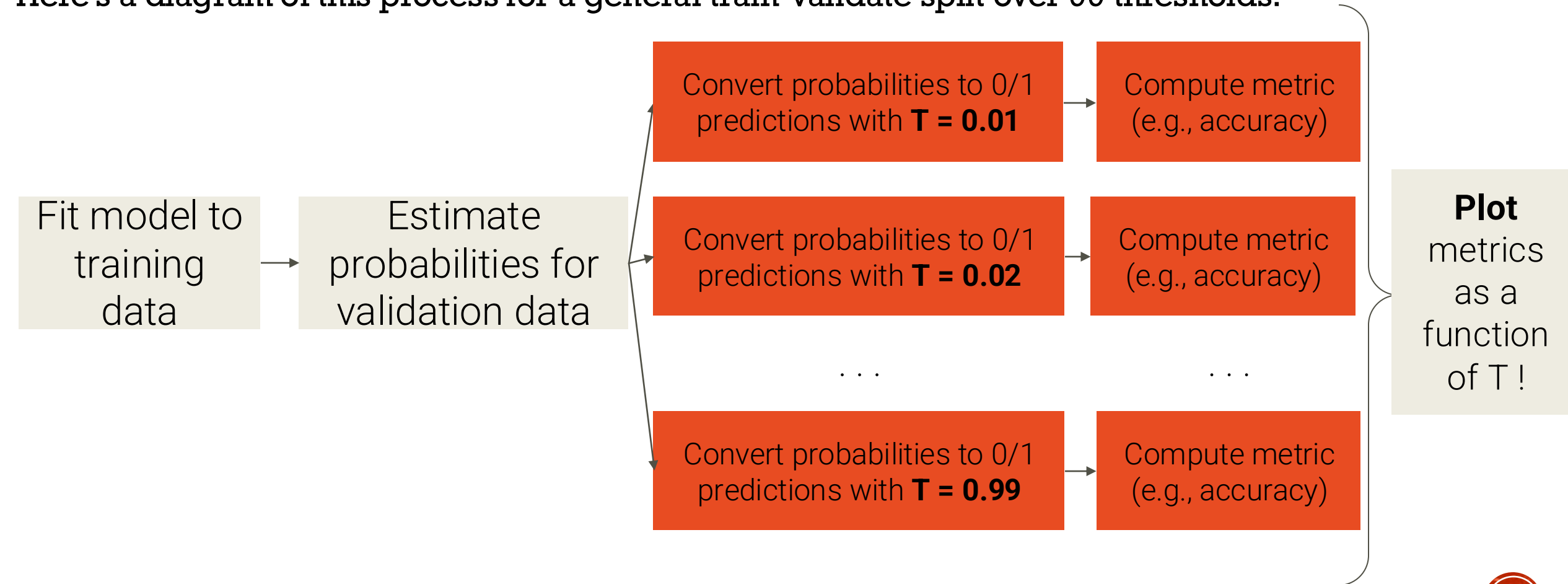
For example, consider credit card fraud:

- FNs (missed fraud) are a lot more costly than FPs (false fraud alert)
- So, it's OK to send a lot false fraud alerts (low precision) as long as we detect most of the actual fraud (high recall).

"SWEEPING THROUGH" THRESHOLDS

In the following slides, we talk about computing metrics over different thresholds (T).

Here's a diagram of this process for a general train-validate split over 99 thresholds:

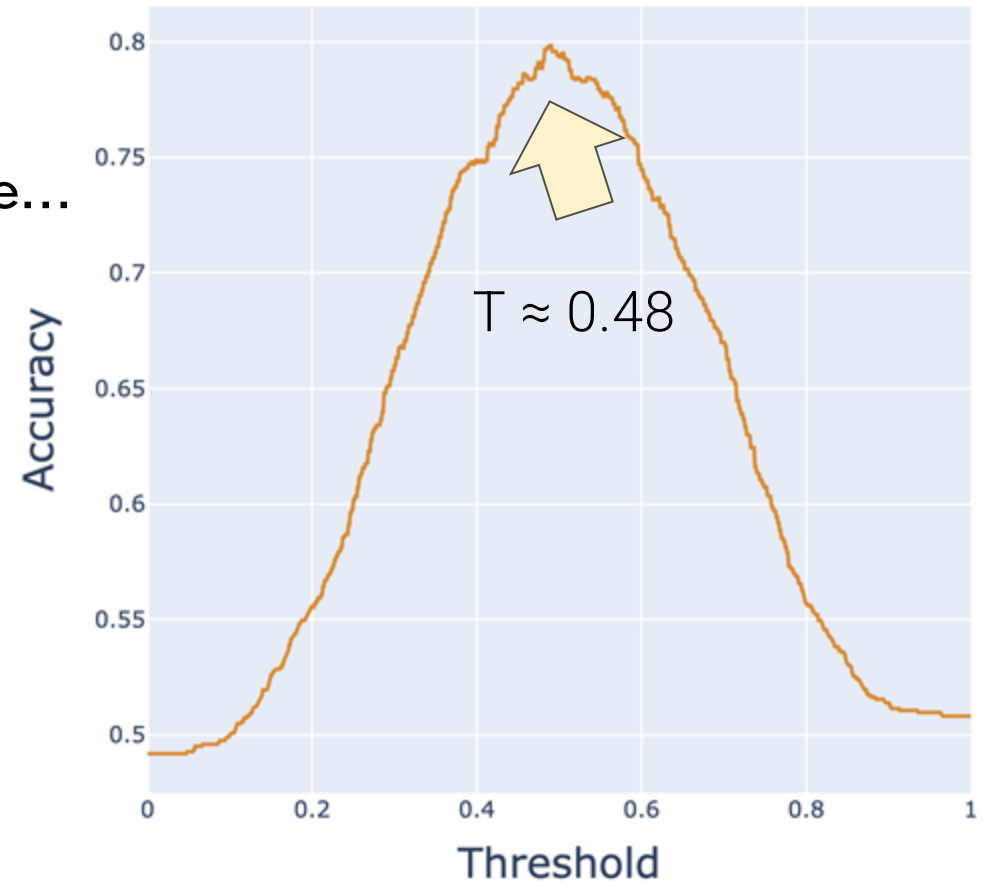


CHOOSING AN ACCURACY THRESHOLD

The choice of threshold T impacts classification performance.

- High T : Most predictions are 0. Lots of FNs.
- Low T : Most predictions are 1. Lots of FPs.

Do we get max accuracy when $T = 0.5$? Not always the case...

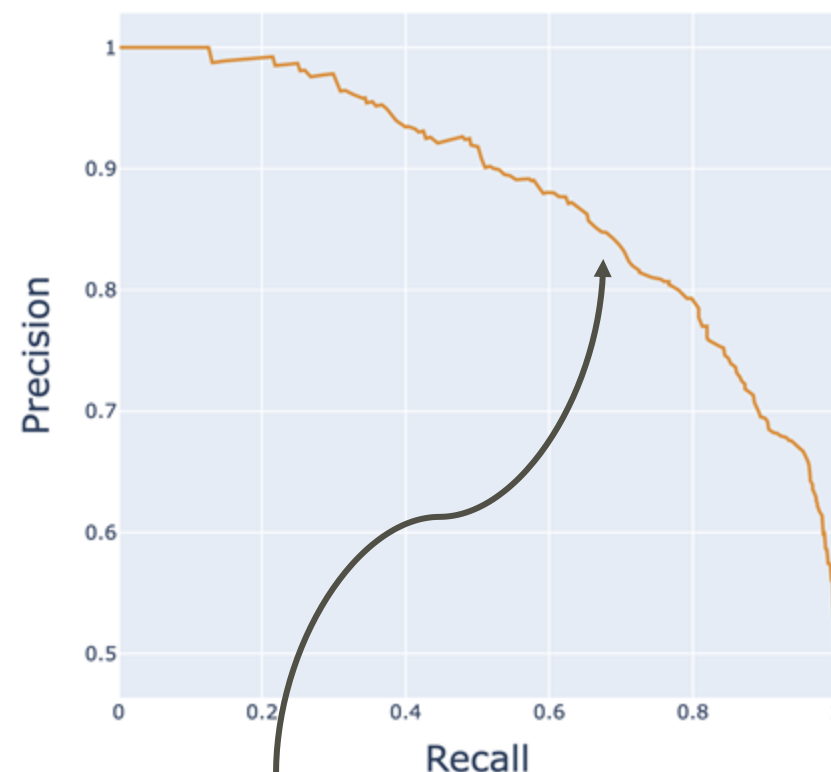
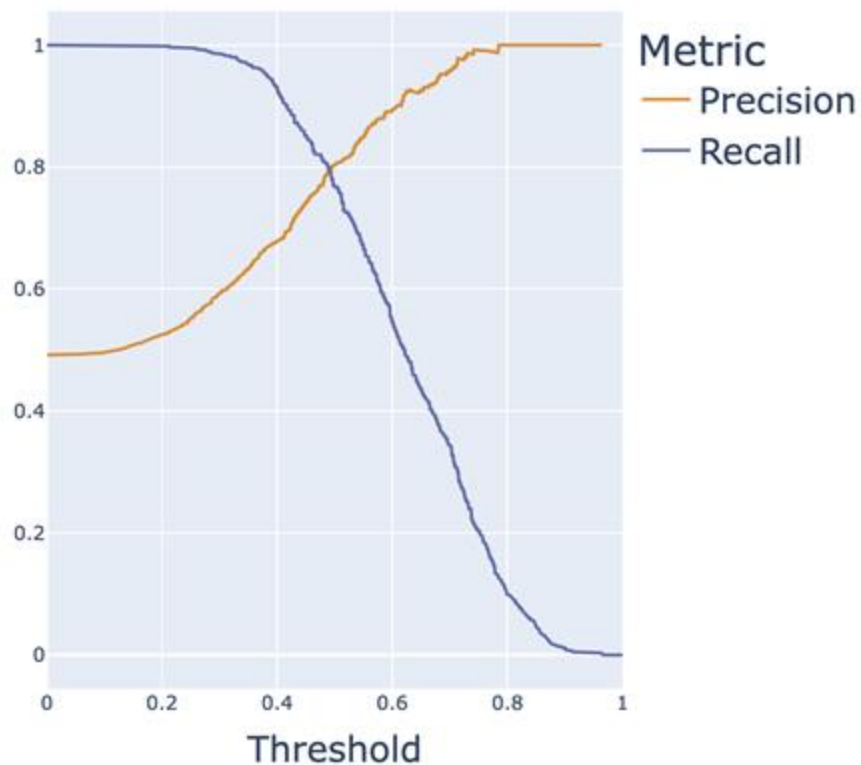


■ See notebook for code snippets.

PRECISION-RECALL CURVES

To construct a precision-recall curve, we:

- 1) Generate predictions for many different thresholds.
- 2) For each threshold, compute precision and recall.



We often choose a threshold that keeps both precision and recall high. But, if the cost of FPs and FNs differs a lot, we may not.

COMPUTING AN F1 SCORE

One way to **balance** precision and recall is to maximize the F_1 Score:

$$F1 \text{ Score} = \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- The **harmonic mean** of precision and recall
- Often used when there is a **large class imbalance**

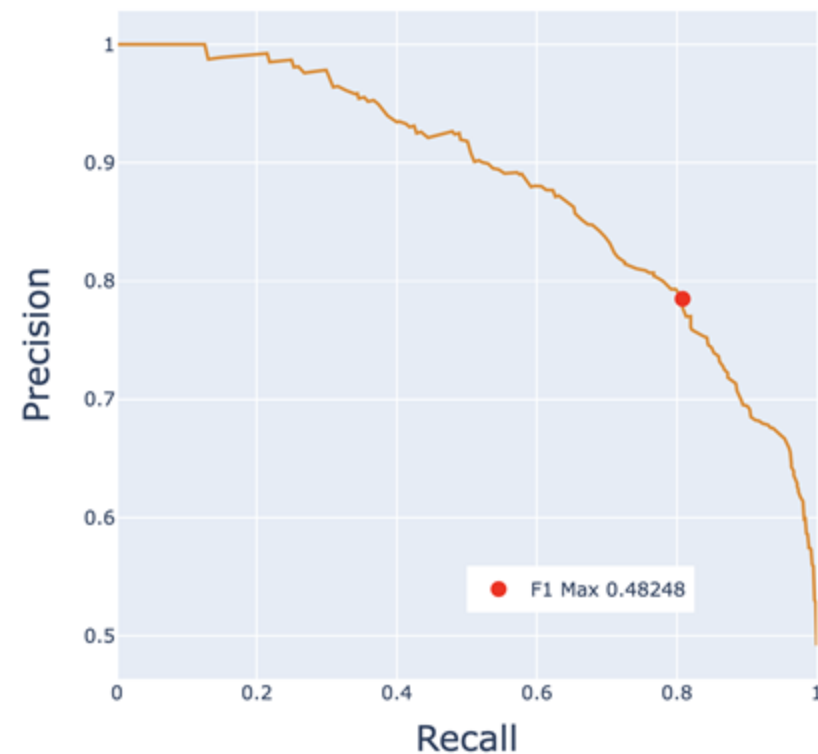
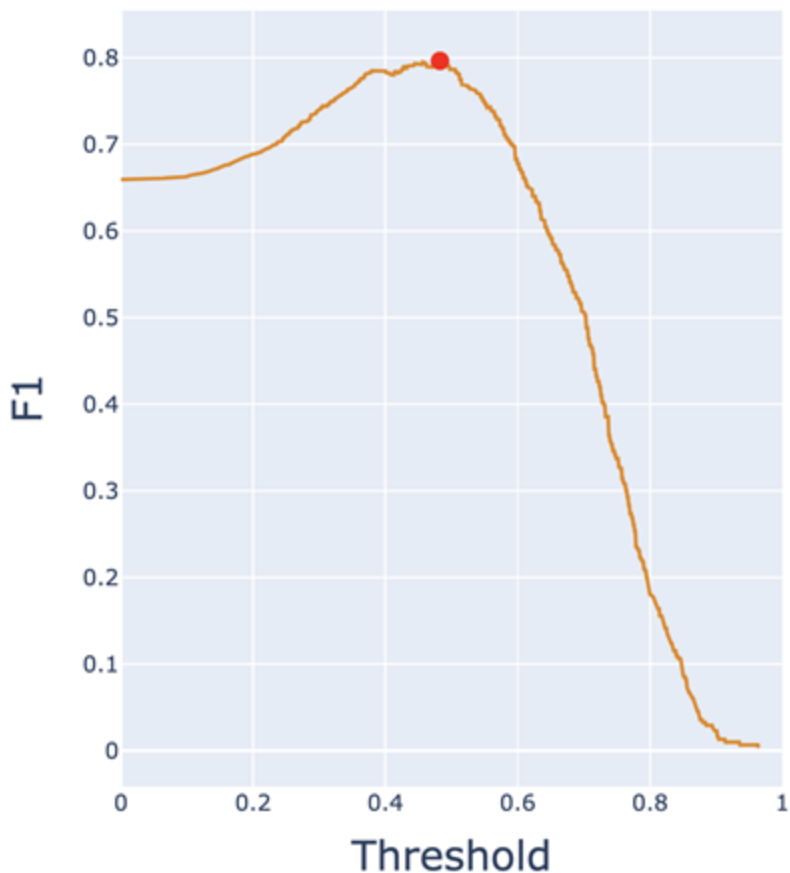
How do you use it?

- Pick the threshold that **maximizes the F_1 score**

Note: If FPs and FNs have different costs, we may not want to balance precision+recall. Think back to credit card fraud!

MAXIMIZING F1 SCORE

We can maximize the F1 score by evaluating the F1 score for many threshold values.



TRUE AND FALSE POSITIVE RATES

Two more performance metrics worth knowing!

$$\text{FPR} = \frac{FP}{FP + TN}$$

False Positive Rate (FPR): Out of all actual 0s, how many did we classify **incorrectly**?

$$\text{TPR} = \frac{TP}{TP + FN}$$

True Positive Rate (TPR): Out of all actual 1s, how many did we classify correctly? **Same as recall.**

		Prediction	
		0	1
Actual	0	TN	FP
	1	FN	TP

Lots of classification metrics out there! All based on TP, TN, FP, FN.

We also have FNR and TNR. Memory tip: The numerator is always the same as the name (FPR → FP), and the denominator is the numerator plus the two opposite letters (FPR → FP + TN).

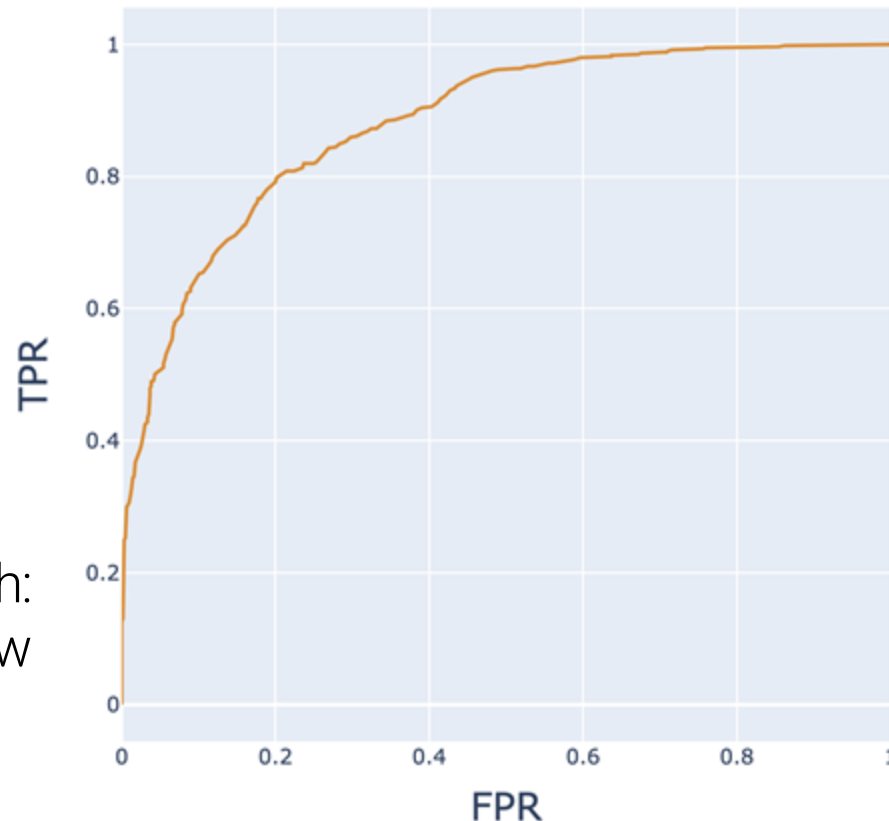
ROC CURVES

We can perform a similar process with FPR and TPR

- 1) Try many thresholds
- 2) Compute the FPR and TPR for each threshold
- 3) Choose a threshold that keeps FPR *low* and TPR *high*

ROC = “receiver operating characteristic” (comes from radar in WWII)

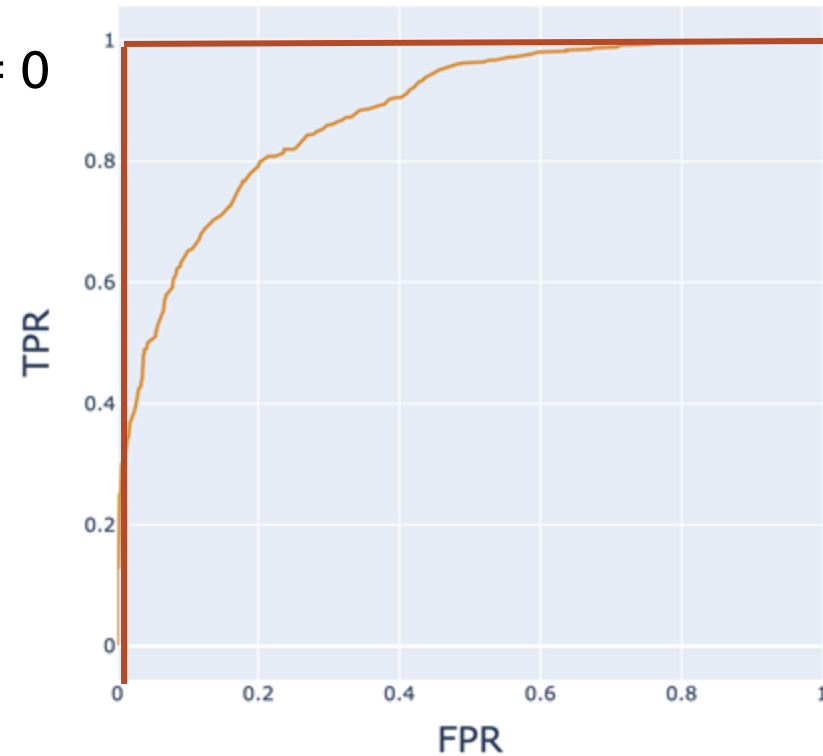
Threshold is high:
FPR low, TPR low
(few positive
predictions)



Threshold is low:
FPR high, TPR high
(many positive
predictions)

ROC CURVES

A **perfect predictor** has $TPR = 1$ and $FPR = 0$

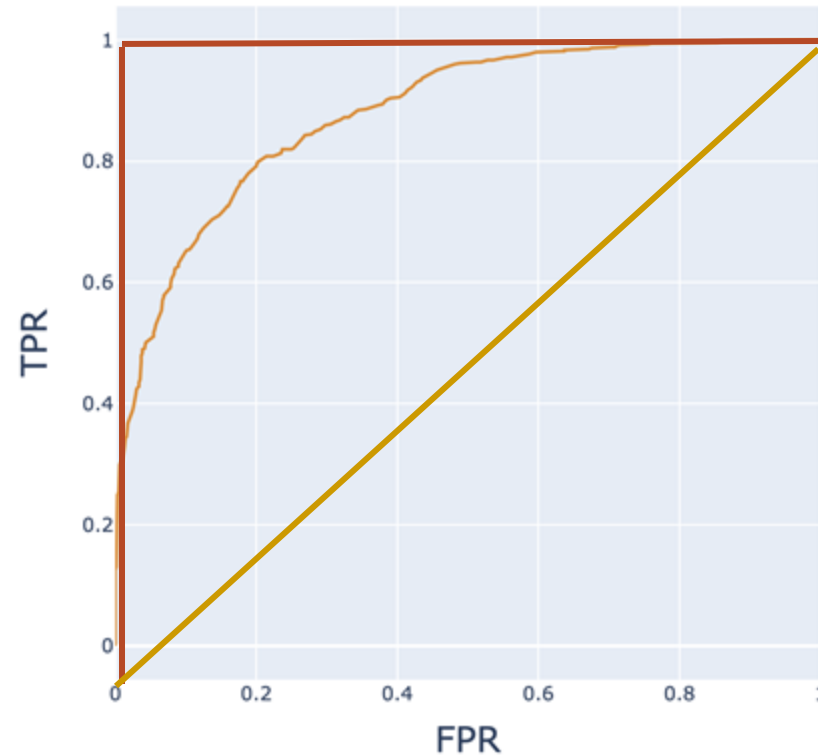


The best **Area Under the ROC Curve (AUC-ROC)** is 1.

Because we want our classifier to be as close as possible to the perfect predictor, we aim to maximize the AUC.

ROC CURVES

A predictor that **predicts randomly** has an AUC-ROC of 0.5



Real-world classifiers have an AUC between 0.5 and 1.

DIFFERENCES BETWEEN AUC-ROC AND PR-AUC

AUC-ROC comes from TPR and FPR.

Precision-recall AUC (PR-AUC) comes from **recall** and **precision**.

TPR=recall, so the only difference is **FPR** and **precision**.

$FPR = FP / \text{Real } 0s$. If your dataset has a lot of real 0s (i.e., imbalanced!), FPR may be **misleadingly low** for a bad classifier.

- So, AUC-ROC is not appropriate for imbalanced datasets. Use PR-AUC instead.

$\text{Precision} = TP / \text{Positives}$. Use PR-AUC if you want to focus on quality of **positive predictions**.

- But, PR-AUC de-emphasizes negative predictions. AUC-ROC is better for **balanced datasets** and when you care **equally** about making good predictions for both classes.

SUMMARY: HOW DO I PICK MY THRESHOLD?

It comes down to the cost of FPs and FNs. Choose threshold that minimizes total cost.

General guidelines when FP and FN costs are similar:

When classes are **balanced** and you care equally about good negative+positive predictions:

- Pick threshold to maximize accuracy (often close to 0.5)
- **Accuracy** and **AUC-ROC** are appropriate metrics to report.

When classes are **imbalanced** or you care more about good **positive** predictions:

- Pick threshold to maximize F_1 score
- **F_1 score** and **Area Under Precision-Recall Curve (PR-AUC)** are appropriate.

Note: **Accuracy** and **F_1 -score** are **threshold-dependent** while AUC metrics are not.

WHAT IS THE “WORST” AUC AND WHY IS IT 0.5?

Best possible AUC = 1. Terrible AUC = 0.5.

- Random predictors have an AUC of around 0.5. Why?

A **random predictor** randomly predicts $P(Y = 1 \mid \mathbf{x})$ to be uniformly between 0 and 1.

$$\text{TPR} = \frac{TP}{TP + FN}$$
$$\text{FPR} = \frac{FP}{FP + TN}$$

On average, if your dataset is size $n_1 + n_0$ (with n_1 true class 1's and n_0 true class 0's):

If $T = 0.5$:

	0	1
0	TN = $0.5 n_0$	FP = $0.5 n_0$
1	FN = $0.5 n_1$	TP = $0.5 n_1$

$$\text{FPR} = 0.5 n_0 / ((0.5 + 0.5) n_0) = 0.5$$

$$\text{TPR} = 0.5 n_1 / ((0.5 + 0.5) n_1) = 0.5$$

Point on ROC curve is
(0.5, 0.5).

If $T = 0.8$:

	0	1
0	TN = $0.8 n_0$	FP = $0.2 n_0$
1	FN = $0.8 n_1$	TP = $0.2 n_1$

$$\text{FPR} = 0.2 n_0 / ((0.2 + 0.8) n_0) = 0.2$$

$$\text{TPR} = 0.2 n_1 / ((0.2 + 0.8) n_1) = 0.2$$

Point on ROC curve is
(0.2, 0.2).

If $T = 0.3$:

	0	1
0	TN = $0.3 n_0$	FP = $0.7 n_0$
1	FN = $0.3 n_1$	TP = $0.7 n_1$

$$\text{FPR} = 0.7 n_0 / ((0.7 + 0.3) n_0) = 0.7$$

$$\text{TPR} = 0.7 n_1 / ((0.7 + 0.3) n_1) = 0.7$$

Point on ROC curve is
(0.7, 0.7).

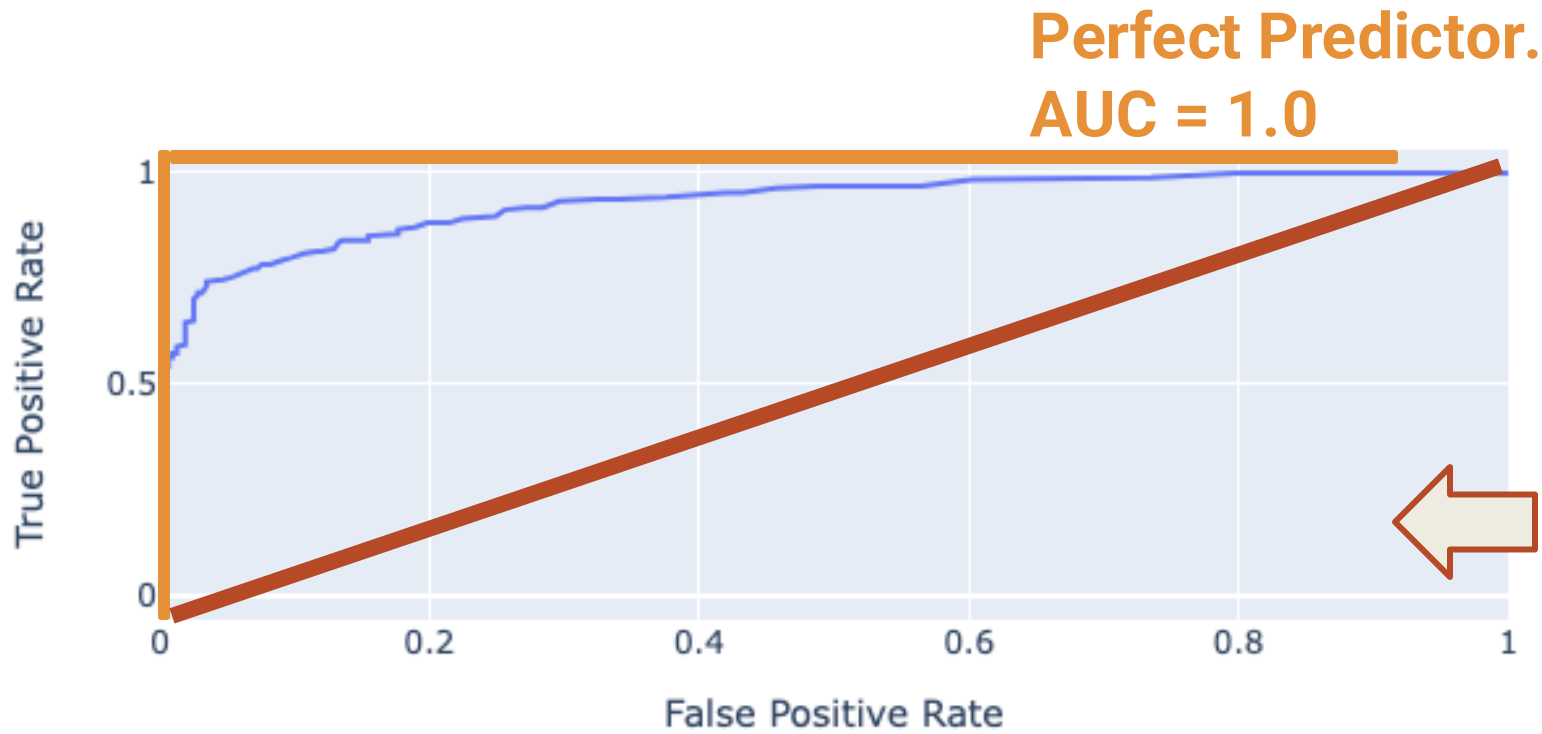
WHAT IS THE “WORST” AUC AND WHY IS IT 0.5?

Best possible AUC = 1. Terrible AUC = 0.5.

- Random predictors have an AUC of around 0.5. Why?

A **random predictor** randomly predicts $P(Y = 1 \mid \mathbf{x})$ to be uniformly between 0 and 1.

$$\text{TPR} = \frac{TP}{TP + FN}$$
$$\text{FPR} = \frac{FP}{FP + TN}$$



Random Predictor.
AUC = 0.5.

Area Under Curve (AUC) of random predictor is the area of this triangle.

READING MATERIAL

- Chapter# 4: Principle of Data Science
- Chapter# 2 & 4: Hands-On ML [Aurélien]